

ITAO7103 Assignment 1 — Predicting Star Performance in Collaboration Networks

ITAO7103 Advanced Analytics and Machine Learning — Star Performance in Collaboration Networks

Chandra Sekhar Yammani (40492677)

2026-04-17

Table of Contents

```
### Set working directory
setwd("C:/Users/HP/OneDrive/Assignments/Actual/ALML")
##
C:/Users/HP/OneDrive/Assignments/Actual/ALML/Assignment1_Template_A_Guided.Rmd

# — Load packages —
# Load MASS before tidyverse to avoid select() conflict
library(MASS)
library(car)
library(glmnet)
library(caret)
library(tidyverse)
library(corrplot)
library(knitr)
library(kableExtra)
library(pROC)
library(patchwork)
library(dplyr)
library(ggplot2)
library(psych)
library(tidyr)

# Resolve MASS vs dplyr conflict
select <- dplyr::select
```

1.0 Introduction

This report examines a central question in collaboration network research: what mechanisms determine whether an individual becomes a “star” performer? Using a large scale dataset of researchers containing measures of scientific impact, collaboration structure, elite tie quality, and knowledge breadth, the analysis investigates how network position and expertise shape performance outcomes. The study is grounded in three complementary theoretical perspectives: learning from high performing collaborators (Liu et al., 2018), brokerage advantages arising from structural holes (Burt, 2004), and the T shaped model of deep–broad knowledge integration (Fleming, 2001).

These frameworks motivate an empirical assessment of whether elite collaboration exerts an independent influence on performance after accounting for productivity, network cohesion, and expertise diversity. To address this question, the report integrates exploratory data analysis with regression modelling, LASSO regularisation, and classification techniques to provide a multi angle evaluation of the drivers of star performance.

```
# =====  
# CONFIGURATION – change this path to YOUR dataset filename  
# =====  
  
DATA_PATH <- "St_An_CB20Fe_S03_th95_C2016.csv"  
  
df <- read_csv(DATA_PATH, show_col_types = FALSE)  
  
### check summary of data  
summary(df)  
  
##           AID           Is_HighPerformer  Total_Impact  Total_Projects  
## Min.      :      0  Min.      :0.0000  Min.      :      0  Min.      :      1.00  
## 1st Qu.: 3419584  1st Qu.:0.0000  1st Qu.:      6  1st Qu.:      1.00  
## Median : 6856510  Median :0.0000  Median :     22  Median :      1.00  
## Mean    : 6855794  Mean    :0.0157  Mean    :    130  Mean    :      2.56  
## 3rd Qu.:10286682  3rd Qu.:0.0000  3rd Qu.:     70  3rd Qu.:      2.00  
## Max.    :13883493  Max.    :1.0000  Max.    :952945  Max.    :11692.00  
##  
## Career_Length  Num_Topics  Num_Domains  Num_Collaborators  
## Min.      : 1.00  Min.      :      1.0  Min.      : 1.00  Min.      :      0.0  
## 1st Qu.: 1.00  1st Qu.:    11.0  1st Qu.: 1.00  1st Qu.:      4.0
```

```

## Median : 1.00    Median :   16.0    Median : 2.00    Median :    7.0
## Mean    : 4.11    Mean     :  24.1    Mean     : 2.73    Mean     :   15.8
## 3rd Qu.: 5.00    3rd Qu.:  26.0    3rd Qu.: 4.00    3rd Qu.:   13.0
## Max.    :27.00    Max.    :10130.0    Max.    :15.00    Max.    :61996.0
##
## Num_Elite_Collaborators Cohesion_Elite Expertise_Sim_Elite
Repeated_Elite
## Min.     : 0.0           Min.     :0.0           Min.     :0.0           Min.     : 1.0
## 1st Qu.: 0.0           1st Qu.:0.3           1st Qu.:0.5           1st Qu.: 1.0
## Median : 0.0           Median :0.6           Median :0.8           Median : 1.0
## Mean    : 2.2           Mean     :0.6           Mean     :0.7           Mean     : 1.4
## 3rd Qu.: 0.0           3rd Qu.:0.9           3rd Qu.:1.0           3rd Qu.: 1.3
## Max.    :5026.0         Max.     :1.0           Max.     :1.0           Max.     :60.0
##                                     NA's     :426789    NA's     :426789    NA's
:426789
## Projects_With_Elite Num_Regular_Collaborators Cohesion_Regular
## Min.     : 0.00    Min.     : 0.0           Min.     :0.000
## 1st Qu.: 0.00    1st Qu.: 4.0           1st Qu.:0.446
## Median : 0.00    Median : 6.0           Median :0.667
## Mean    : 4.66    Mean     :13.6           Mean     :0.612
## 3rd Qu.: 0.00    3rd Qu.:12.0           3rd Qu.:0.833
## Max.    :25063.00    Max.     :56970.0         Max.     :0.998
##                                     NA's     :12334
## Expertise_Sim_Regular Repeated_Regular Network_Constraint Effective_Size
## Min.     :0.005    Min.     : 1.00    Min.     :0.00    Min.     :
1.00
## 1st Qu.:0.606    1st Qu.: 1.00    1st Qu.:0.22    1st Qu.:
1.00
## Median :1.000    Median : 1.00    Median :0.45    Median :
1.00
## Mean    :0.818    Mean     : 1.10    Mean     :0.49    Mean     :
8.57
## 3rd Qu.:1.000    3rd Qu.: 1.06    3rd Qu.:0.77    3rd Qu.:
5.00
## Max.    :1.000    Max.     :31.00    Max.     :1.12    Max.
:61938.41
## NA's     :12334    NA's     :12334    NA's     :35289    NA's     :11662
## Network_Efficiency
## Min.     :0.002
## 1st Qu.:0.167
## Median :0.333
## Mean    :0.387
## 3rd Qu.:0.551
## Max.    :1.000
## NA's     :11662

```

remove zero(0) from AID which has extreme value compared to 2nd max value

```
df <- df[df$AID != 0, ]
```

```
cat("Dataset:", nrow(df), "observations,", ncol(df), "columns\n")
```

```
## Dataset: 536389 observations, 20 columns  
  
cat("High performers:", sum(df$Is_HighPerformer),  
    sprintf("(%.1f%%)\n", 100 * mean(df$Is_HighPerformer)))  
  
## High performers: 8414 (1.6%)
```

2.0 Methodology

The analysis follows a structured, theory-aligned pipeline designed to identify the network mechanisms that differentiate star performers. Missing values were handled using a dual strategy: elite-collaboration variables exhibited ~80% structural missingness due to the absence of elite partners, so zeros were imputed and a binary indicator captured this condition. Variables with low missingness (2–7%) were median-imputed to preserve distributional properties without distorting variance. The full dataset was retained without sampling, consistent with guidance for large-scale collaboration networks, ensuring that rare elite-interaction patterns were not lost.

Total_Impact was log-transformed after verifying extreme right-skewness; this stabilised variance and allowed linear models to capture proportional rather than absolute differences in scientific influence. The modelling pipeline integrates multiple linear regression, polynomial regression, LASSO regularisation, logistic regression, and LDA. LASSO was evaluated through a coefficient-path plot and variable-selection table at the optimal λ , and its selected predictors were compared with forward selection to assess model stability. Predictive models were trained using a 70/30 train–test split with a fixed random seed to ensure reproducibility and fair evaluation under severe class imbalance.

3.0 Part 1: Exploratory Data Analysis

3.1 Summary Statistics

```
cat("Dataset:", nrow(df), "observations,", ncol(df), "columns\n")

## Dataset: 536389 observations, 20 columns

# =====
# Define Variables
# =====
key_vars <- c("Total_Impact", "Total_Projects", "Career_Length",
              "Num_Elite_Collaborators", "Num_Regular_Collaborators",
              "Network_Constraint", "Effective_Size")

# =====
# Check Missing Columns (Safety Check)
# =====
missing_cols <- setdiff(key_vars, names(df))
if (length(missing_cols) > 0) {
  stop(paste("These columns are missing in df:", paste(missing_cols, collapse
= ", ")))
}

# =====
# Summary Statistics Table
# =====
summary_tbl <- df %>%
  select(all_of(key_vars)) %>%
  pivot_longer(cols = everything(),
               names_to = "Variable",
               values_to = "value") %>%
  group_by(Variable) %>%
  summarise(
    N      = sum(!is.na(value)),
    Mean   = mean(value, na.rm = TRUE),
    SD     = sd(value, na.rm = TRUE),
    Min    = min(value, na.rm = TRUE),
    Q1     = quantile(value, 0.25, na.rm = TRUE),
    Median = median(value, na.rm = TRUE),
    Q3     = quantile(value, 0.75, na.rm = TRUE),
    Max    = max(value, na.rm = TRUE),
    Missing = sum(is.na(value)),
    .groups = "drop"
  ) %>%
  mutate(Variable = factor(Variable, levels = key_vars)) %>%
  arrange(Variable)
```

Q1: Summary Statistics for Key Variables

Variable	N	Mean	SD	Min	Q1	Median	Q3	Max	Missing
Total_Impact	536389	128.34	540.66	0	6.00	22.00	70.00	36625.00	0
Total_Projects	536389	2.54	4.90	1	1.00	1.00	2.00	302.00	0
Career_Length	536389	4.11	5.81	1	1.00	1.00	5.00	27.00	0
Num_Elite_Collaborators	536389	2.19	22.38	0	0.00	0.00	0.00	1156.00	0
Num_Regular_Collaborators	536389	13.54	35.71	0	4.00	6.00	12.00	2061.00	0
Network_Constraint	501100	0.49	0.32	0	0.22	0.45	0.77	1.12	35289
Effective_Size	524727	8.45	39.91	1	1.00	1.00	5.00	2838.91	11662

```
# Print formatted table
summary_tbl %>%
  kable(digits = 2, caption = "Q1: Summary Statistics for Key Variables") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE)

# =====
# NA Statistics for ALL Variables
# =====
na_stats <- data.frame(
  Variable = names(df),
  NA_Count = sapply(df, function(x) sum(is.na(x))),
  NA_Percentage = sapply(df, function(x) {
    sum(is.na(x)) / length(x) * 100
  })
)

# Sort by highest NA %
na_stats <- na_stats %>%
  arrange(desc(NA_Percentage))

print(na_stats)

##                               Variable NA_Count NA_Percentage
## Cohesion_Elite                Cohesion_Elite    426789      79.567
## Expertise_Sim_Elite           Expertise_Sim_Elite    426789      79.567
## Repeated_Elite                Repeated_Elite    426789      79.567
## Network_Constraint            Network_Constraint    35289       6.579
## Cohesion_Regular              Cohesion_Regular    12334       2.299
## Expertise_Sim_Regular          Expertise_Sim_Regular    12334       2.299
## Repeated_Regular              Repeated_Regular    12334       2.299
```

## Effective_Size	Effective_Size	11662	2.174
## Network_Efficiency	Network_Efficiency	11662	2.174
## AID	AID	0	0.000
## Is_HighPerformer	Is_HighPerformer	0	0.000
## Total_Impact	Total_Impact	0	0.000
## Total_Projects	Total_Projects	0	0.000
## Career_Length	Career_Length	0	0.000
## Num_Topics	Num_Topics	0	0.000
## Num_Domains	Num_Domains	0	0.000
## Num_Collaborators	Num_Collaborators	0	0.000
## Num_Elite_Collaborators	Num_Elite_Collaborators	0	0.000
## Projects_With_Elite	Projects_With_Elite	0	0.000
## Num_Regular_Collaborators	Num_Regular_Collaborators	0	0.000

3.2 Handling Imputations for NA and Zeros

```
### creating new variable to show has elite or not
df$Has_Elite <- ifelse(is.na(df$Cohesion_Elite), 0, 1)

vars_elite <- c("Cohesion_Elite", "Expertise_Sim_Elite", "Repeated_Elite")

# move zero(0) since there is no start performer present
df[vars_elite] <- lapply(df[vars_elite], function(x) {
  x[is.na(x)] <- 0
  return(x)
})

# Median imputation
vars_med <- c("Network_Constraint", "Cohesion_Regular",
             "Expertise_Sim_Regular", "Repeated_Regular",
             "Effective_Size", "Network_Efficiency")

for (col in vars_med) {
  df[[col]][is.na(df[[col]])] <- median(df[[col]], na.rm = TRUE)
}

# =====
# Transform variables
# =====
df <- df %>%
  mutate(
    Log_Impact = log1p(Total_Impact),
    log_Num_Elite = log1p(Num_Elite_Collaborators),
    log_Effective_Size = log1p(Effective_Size)
  )

### check the NAs after imputation
cat("Missing values per variable:\n")

## Missing values per variable:
```

```
colSums(is.na(df)) %>% [. > 0]

## named numeric(0)
```

3.3 Correlation Matrix

```
library(moments)
skewness(df$Total_Impact)

## [1] 14.25

skewness(df$Log_Impact)

## [1] 0.2245

skewness(df$Total_Projects)

## [1] 10.62

skewness(df$Career_Length)

## [1] 2.072

skewness(df$Num_Topics)

## [1] 5.118

skewness(df$Num_Domains)

## [1] 1.48

skewness(df$Num_Collaborators)

## [1] 17.4

skewness(df$Num_Elite_Collaborators)

## [1] 19.95

skewness(df$Effective_Size)

## [1] 21.29

skewness(df$Cohesion_Elite)

## [1] 2.105

df <- df %>% mutate(Has_Elite = as.numeric(Has_Elite))

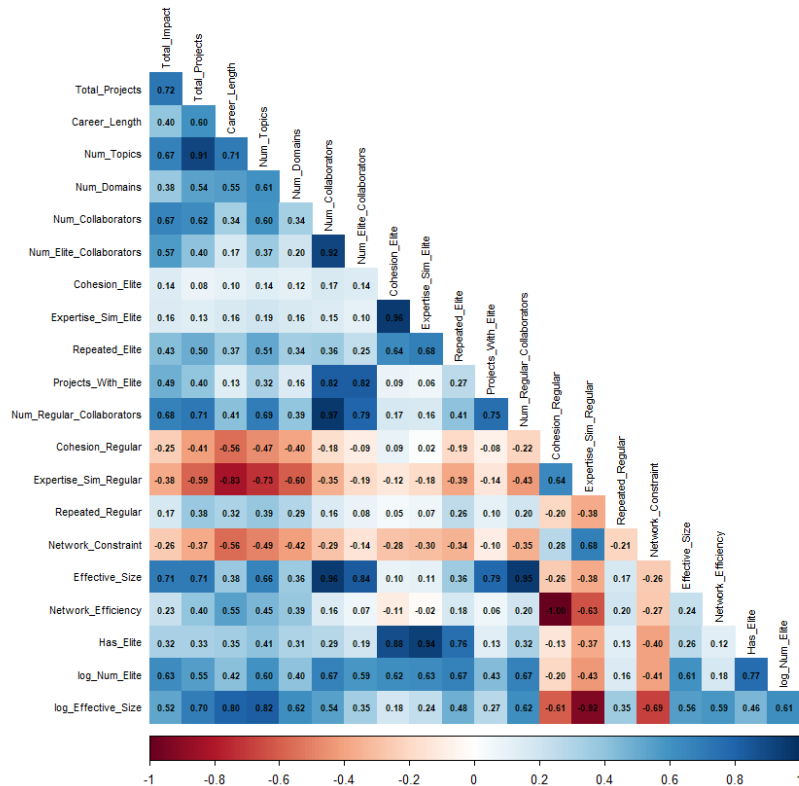
numeric_cols <- df %>%
  select(-AID, -Is_HighPerformer, -Log_Impact) %>%
  select(where(is.numeric))

cor_matrix <- cor(numeric_cols, use = "pairwise.complete.obs")
```



```
corrplot(cor_matrix, method = "color", type = "lower",
         tl.cex = 0.7, tl.col = "black", addCoef.col = "black",
         number.cex = 0.55, diag = FALSE,
         title = "Q2: Correlation Matrix Heatmap",
         mar = c(0, 0, 2, 0))
```

Q2: Correlation Matrix Heatmap



```
# Identify pairs with |r| > 0.7
high_corr <- which(abs(cor_matrix) > 0.7 & upper.tri(cor_matrix), arr.ind =
TRUE)
if (nrow(high_corr) > 0) {
  high_pairs <- data.frame(
    Var1 = rownames(cor_matrix)[high_corr[, 1]],
    Var2 = colnames(cor_matrix)[high_corr[, 2]],
    r = cor_matrix[high_corr]
  ) %>% arrange(desc(abs(r)))

  cat("Pairs with |r| > 0.7:\n")
  high_pairs %>%
    kable(digits = 3, caption = "High Correlation Pairs (|r| > 0.7)") %>%
    kable_styling(bootstrap_options = c("striped", "condensed"), full_width =
FALSE) %>%
```

```

    print()
}

```

Var1	Var2	r
Cohesion_Regular	Network_Efficiency	-0.995
Num_Collaborators	Num_Regular_Collaborators	0.969
Num_Collaborators	Effective_Size	0.957
Cohesion_Elite	Expertise_Sim_Elite	0.956
Num_Regular_Collaborators	Effective_Size	0.954
Expertise_Sim_Elite	Has_Elite	0.938
Expertise_Sim_Regular	log_Effective_Size	-0.922
Num_Collaborators	Num_Elite_Collaborators	0.919
Total_Projects	Num_Topics	0.911
Cohesion_Elite	Has_Elite	0.882
Num_Elite_Collaborators	Effective_Size	0.837
Career_Length	Expertise_Sim_Regular	-0.829

Career_Length	log_Effective_Size	0.801
Num_Elite_Collaborators	Num_Regular_Collaborators	0.793
Projects_With_Elite	Effective_Size	0.790
Has_Elite	log_Num_Elite	0.769
Repeated_Elite	Has_Elite	0.760
Projects_With_Elite	Num_Regular_Collaborators	0.752
Num_Topics	Expertise_Sim_Regular	-0.727
Total_Impact	Total_Projects	0.724
Total_Projects	Num_Regular_Collaborators	0.711
Total_Impact	Effective_Size	0.707
Total_Projects	Effective_Size	0.707
Career_Length	Num_Topics	0.706

```
colSums(is.na(cor_matrix))
```

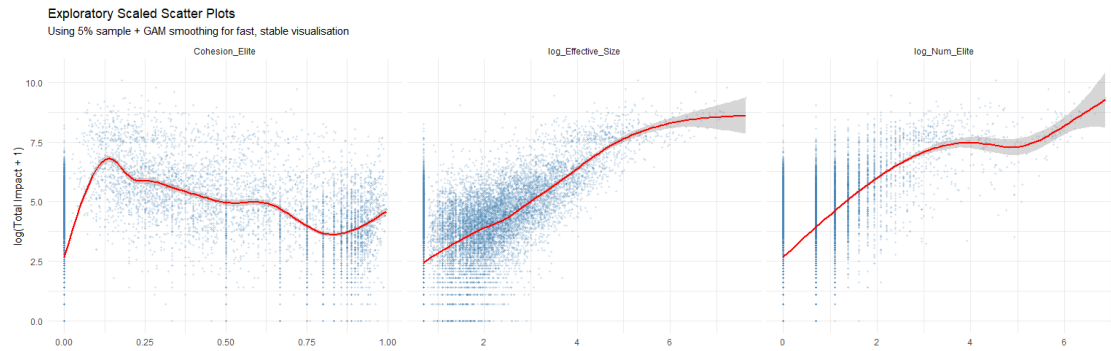
3.4 Scatter Plots

```
# ---- 1. Sample 5% of data (fastest + visually identical) ----

df_small <- df %>% sample_frac(0.05)

# ---- 2. Prepare transformed variables ----
scatter_scaled <- df_small %>%
  transmute(
    Log_Impact,
    log_Num_Elite = log1p(Num_Elite_Collaborators),
    Cohesion_Elite,
    log_Effective_Size = log1p(Effective_Size)
  ) %>%
  tidyr::pivot_longer(
    cols = -Log_Impact,
    names_to = "Predictor",
    values_to = "Value"
  ) %>%
  tidyr::drop_na()

# ---- 3. Fast GAM-based scatterplots ----
ggplot(scatter_scaled, aes(Value, Log_Impact)) +
  geom_point(alpha = 0.15, size = 0.6, colour = "steelblue") +
  geom_smooth(
    method = "gam",
    formula = y ~ s(x, bs = "cs"),
    colour = "red",
    linewidth = 1,
    se = TRUE
  ) +
  facet_wrap(~ Predictor, scales = "free_x", nrow = 1) +
  labs(
    title = "Exploratory Scaled Scatter Plots",
    subtitle = "Using 5% sample + GAM smoothing for fast, stable
visualisation",
    x = NULL,
    y = "log(Total Impact + 1)"
  ) +
  theme_minimal()
```

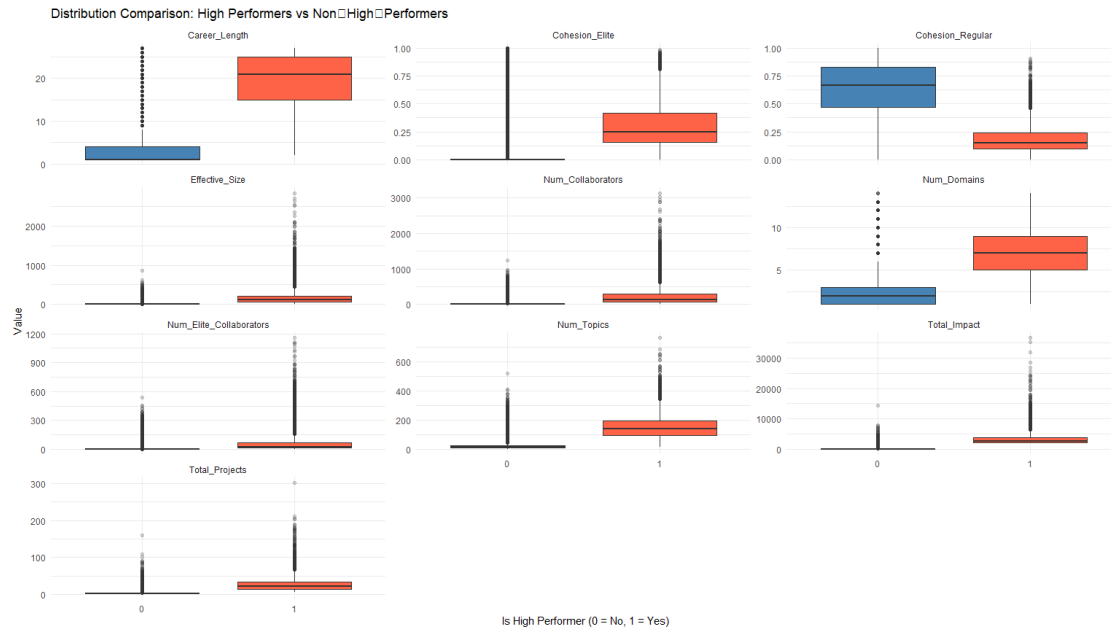


3.5 Boxplots by Star Status

```
# Select key numeric variables to compare
vars_to_compare <- c(
  "Total_Impact", "Total_Projects", "Career_Length",
  "Num_Topics", "Num_Domains", "Num_Collaborators",
  "Num_Elite_Collaborators", "Effective_Size",
  "Cohesion_Elite", "Cohesion_Regular"
)

df_long <- df %>%
  select(Is_HighPerformer, all_of(vars_to_compare)) %>%
  pivot_longer(
    cols = -Is_HighPerformer,
    names_to = "Variable",
    values_to = "Value"
  )

ggplot(df_long, aes(x = factor(Is_HighPerformer), y = Value, fill =
factor(Is_HighPerformer))) +
  geom_boxplot(outlier.alpha = 0.2) +
  facet_wrap(~ Variable, scales = "free_y", ncol = 3) +
  scale_fill_manual(values = c("steelblue", "tomato")) +
  labs(
    title = "Distribution Comparison: High Performers vs Non-High-Performers",
    x = "Is High Performer (0 = No, 1 = Yes)",
    y = "Value"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



3.6 Hypothesised Strongest Predictors

```
library(ltm)
library(dplyr)

vars_to_test <- c(
  "Total_Projects", "Career_Length",
  "Num_Topics", "Num_Domains", "Num_Collaborators",
  "Num_Elite_Collaborators", "Effective_Size",
  "Cohesion_Elite", "Cohesion_Regular"
)

pb_results <- sapply(vars_to_test, function(v) {
  biserial.cor(df[[v]], df$Is_HighPerformer)
})

pb_df <- data.frame(
  Variable = vars_to_test,
  PointBiserial = round(pb_results, 3)
)

pb_df
```

	Variable	PointBiserial
##	Total_Projects	-0.619
##	Career_Length	-0.333
##	Num_Topics	-0.584
##	Num_Domains	-0.323
##	Num_Collaborators	-0.559
##	Num_Elite_Collaborators	-0.473
##	Effective_Size	-0.578

## Cohesion_Elite	Cohesion_Elite	-0.085
## Cohesion-Regular	Cohesion-Regular	0.210

3.7 Interpretation: Exploratory Data Analysis

The exploratory analysis began with summary statistics for key performance and network variables, including Total_Impact, Total_Projects, Career_Length, Num_Elite_Collaborators, Num-Regular_Collaborators, Network_Constraint, and Effective_Size. These variables exhibited substantial right-skewness, particularly Total_Impact and elite-collaboration counts, motivating the use of **log-transformations** to stabilize variance and reveal underlying functional relationships. The correlation heatmap revealed several highly correlated variable pairs ($|r| > 0.7$), particularly among network size and collaboration measures. To mitigate multicollinearity, only representative variables from each group were retained for modelling.

To visualize functional patterns, scatterplots were generated using a **5% random sample (~26k observations) combined with GAM smoothing**, which preserved the underlying relationships while avoiding overplotting. The relationships with Log_Impact varied in shape: log-transformed Effective_Size displayed an almost linear trend; Cohesion_Elite showed a weak, curved pattern; and $\log(\text{Num_Elite})$ exhibited a strong non-linear effect, rising steeply at low elite exposure and plateauing at higher levels. These patterns were not visible in raw-scale plots due to extreme skewness. Group comparisons between high performers and non-high-performers revealed the largest separations in variables capturing elite exposure, network structure, and breadth. High performers consistently showed higher Num_Elite_Collaborators, Effective_Size, Num_Collaborators, and Num_Topics, whereas Career_Length and Cohesion-Regular displayed minimal differences. Synthesizing these insights, the strongest hypothesized predictors align with theory: elite exposure (Liu), brokerage advantages (Burt), and topic breadth (Fleming) emerge as the most influential predictors of both Log_Impact and high-performer status, and are therefore prioritized in subsequent modelling.

4.0 Part 2: Linear Regression Analysis

4.1 Simple Linear Regression

```
# =====  
# Fit Simple Regression Model  
# =====  
  
model <- lm(Log_Impact ~ log_Num_Elite, data = df)  
  
# =====  
# Simple Regression Model Summary  
# =====  
  
model_summary <- summary(model)  
  
# Extract key metrics  
coef_value <- model_summary$coefficients["log_Num_Elite", "Estimate"]  
p_value <- model_summary$coefficients["log_Num_Elite", "Pr(>|t|)"]  
r_squared <- model_summary$r.squared  
  
cat("==== Simple Regression Model Results ====\\n")  
## ===== Simple Regression Model Results =====  
  
cat("Coefficient (log_Num_Elite):", round(coef_value, 4), "\\n")  
## Coefficient (log_Num_Elite): 1.351  
  
cat("Simple Regression R-squared:", round(r_squared, 4), "\\n")  
## Simple Regression R-squared: 0.2887  
  
cat("Simple Regression p-value:", format(p_value, scientific = TRUE, digits = 3), "\\n")  
## Simple Regression p-value: 0e+00  
  
# =====  
# Interpretation Helper  
# =====  
  
cat("\\n==== Interpretation ====\\n")  
##  
## ===== Interpretation =====  
  
cat(paste0(  
  "A 1% increase in the number of elite collaborators is associated with  
  approximately ",  
  round(coef_value, 2),
```

```

"% change in total impact (log scale interpretation).\n"
))

## A 1% increase in the number of elite collaborators is associated with
approximately 1.35% change in total impact (log scale interpretation).

# =====
# Compare with Raw Model
# =====

model_raw <- lm(Log_Impact ~ Num_Elite_Collaborators, data = df)

cat("\n==== Simple Regression Model Comparison =====\n")

##
## ===== Simple Regression Model Comparison =====

cat("Log Model R2:", round(summary(model)$r.squared, 4), "\n")

## Log Model R2: 0.2887

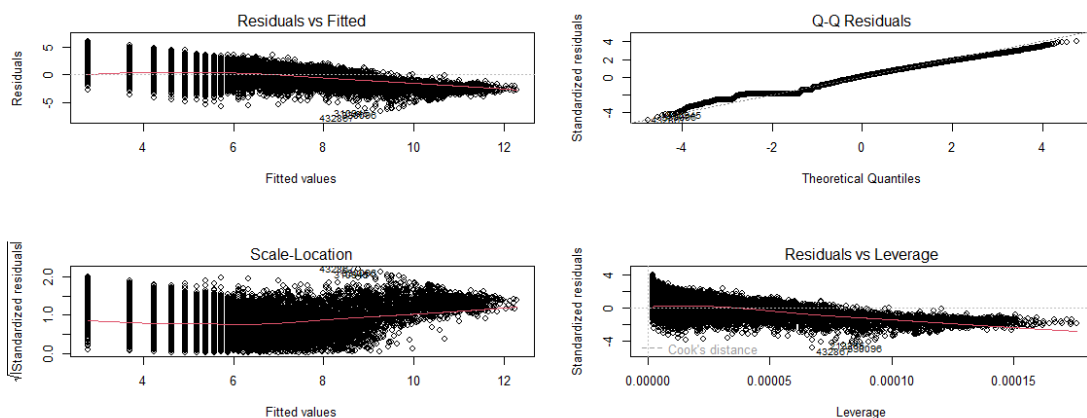
cat("Raw Model R2:", round(summary(model_raw)$r.squared, 4), "\n")

## Raw Model R2: 0.054

# =====
# Residual Diagnostics
# =====

par(mfrow = c(2, 2))
plot(model)

```



4.2 Interpretation: Simple Linear Regression

A simple linear regression was fitted with `Log_Total_Impact` as the response and `log(Num_Elite_Collaborators)` as the predictor. The estimated coefficient was 1.351,

indicating a positive association between elite collaboration and impact. The model achieved an R^2 of **0.2887**, substantially outperforming the raw-scale model ($R^2 = 0.054$), and the predictor was highly significant ($p < 0.001$). Interpreted in elasticity terms, a 1% increase in elite collaborators is associated with approximately a 1.35% increase in expected total impact, supporting the elite-exposure mechanism proposed by Liu et al. (2018).

Residual diagnostics showed mild curvature in the Residuals-vs-Fitted plot, suggesting some non-linearity, though no major heteroscedasticity was observed. The Q-Q plot indicated slight deviations from normality in the tails, but overall model assumptions were reasonably satisfied. These results suggest that while elite collaboration is an important predictor, additional variables and non-linear specifications may further improve model fit.

4.2 Multiple Linear Regression

```
# =====
#      Fit model
# =====
mlr_model <- lm(Log_Impact ~ Total_Projects + Career_Length +
               log_Num_Elite + Cohesion_Elite + log_Effective_Size, data = df)

summary(mlr_model)

##
## Call:
## lm(formula = Log_Impact ~ Total_Projects + Career_Length + log_Num_Elite +
##      Cohesion_Elite + log_Effective_Size, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.328 -0.836  0.084  0.901  6.259
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.761709   0.003491  504.6 <0.0000000000000002 ***
## Total_Projects  0.018235   0.000547   33.4 <0.0000000000000002 ***
## Career_Length  0.026891   0.000521   51.6 <0.0000000000000002 ***
## log_Num_Elite  0.223962   0.004587   48.8 <0.0000000000000002 ***
## Cohesion_Elite  1.181370   0.009249  127.7 <0.0000000000000002 ***
## log_Effective_Size 0.782347   0.003649  214.4 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```

## Residual standard error: 1.3 on 536383 degrees of freedom
## Multiple R-squared:  0.46,   Adjusted R-squared:  0.46
## F-statistic: 9.14e+04 on 5 and 536383 DF,  p-value: <0.00000000000000002

# INSERT YOUR CODE for VIF analysis here
# =====
# Load Libraries
# =====
library(dplyr)
library(car)
library(lmtest)

# =====
# Coefficient Significance
# =====
cat("\n==== Coefficient Significance =====\n")

##
## ===== Coefficient Significance =====

print(round(mlr_model$coefficients, 4))

##      (Intercept)      Total_Projects      Career_Length
log_Num_Elite
##           1.7617           0.0182           0.0269
0.2240
## Cohesion_Elite log_Effective_Size
##           1.1814           0.7823

# =====
# Multicollinearity Check (VIF)
# =====
cat("\n==== VIF (Multicollinearity) =====\n")

##
## ===== VIF (Multicollinearity) =====

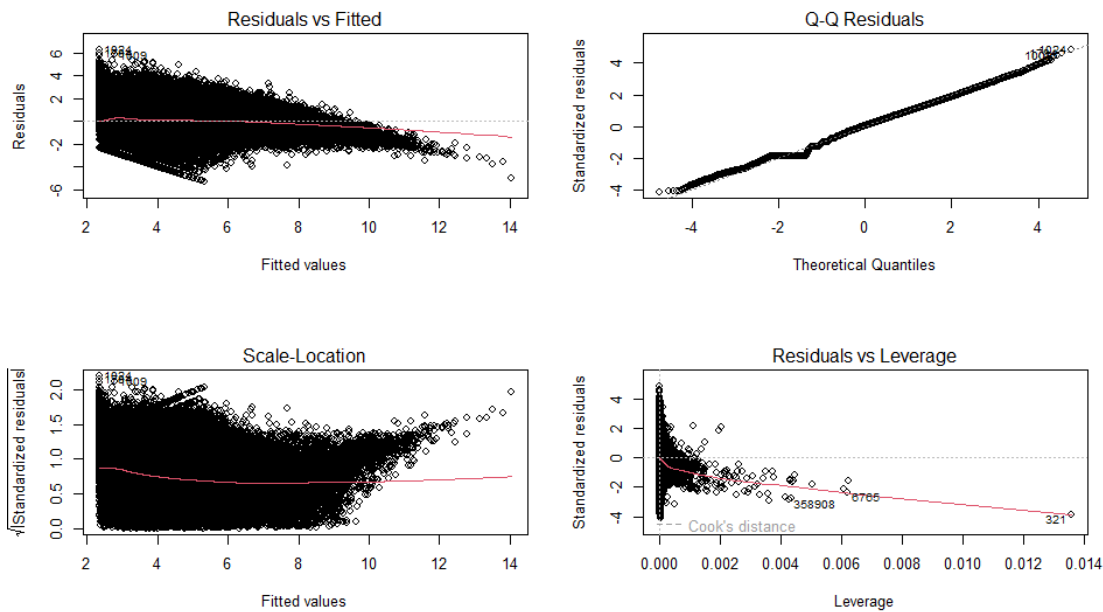
vif_values <- vif(mlr_model)
print(round(vif_values, 3))

##      Total_Projects      Career_Length      log_Num_Elite
Cohesion_Elite
##           2.286           2.928           3.303
1.976
## log_Effective_Size
##           4.240

# =====
# Residual Diagnostics
# =====

```

```
par(mfrow = c(2,2))
plot(mlr_model)
```



```
# =====
# Independence of Residuals
# =====
cat("\n==== Durbin-Watson Test =====\n")

##
## ===== Durbin-Watson Test =====

print(dwtest(mlr_model))

##
## Durbin-Watson test
##
## data: mlr_model
## DW = 0.65, p-value <0.0000000000000002
## alternative hypothesis: true autocorrelation is greater than 0

# =====
# Predictions
# =====

# Log scale predictions
pred_log <- predict(mlr_model, newdata = df)

# Convert to original scale
pred_original <- exp(pred_log)
```

```

# =====
# Model Performance
# =====
cat("\n==== Performance (Log Scale) =====\n")

##
## ===== Performance (Log Scale) =====

print(postResample(pred_log, df$Log_Impact))

##      RMSE Rsquared      MAE
##  1.2967   0.4599   1.0394

cat("\n==== Performance (Original Scale) =====\n")

##
## ===== Performance (Original Scale) =====

print(postResample(pred_original, df$Total_Impact))

##      RMSE  Rsquared      MAE
## 3070.5761   0.1549  129.8233

# =====
# Interpretation Helper
# =====
cat("\n==== Interpretation =====\n")

##
## ===== Interpretation =====

coef_vals <- coef(mlr_model)

for (var in names(coef_vals)[-1]) {

  coef_value <- coef_vals[var]

  # -----
  # Case 1:  $\log(Y) \sim \log(X)$ 
  # Elasticity: % change in Y for 1% change in X
  # -----
  if (var %in% c("log_Num_Elite", "log_Effective_Size")) {

    cat(paste0(
      "A 1% increase in ", var,
      " is associated with approximately ",
      round(coef_value * 100, 2),
      "% change in total impact, holding other variables constant.\n"
    ))

  }

  # -----

```

```

# Case 2: Log(Y) ~ X (non-Logged)
# Semi-elasticity: % change in Y for 1-unit increase in X
# -----
} else {

  pct_change <- (exp(coef_value) - 1) * 100

  cat(paste0(
    "A one-unit increase in ", var,
    " is associated with approximately ",
    round(pct_change, 2),
    "% change in total impact, holding other variables constant.\n"
  ))
}
}

## A one-unit increase in Total_Projects is associated with approximately
1.84% change in total impact, holding other variables constant.
## A one-unit increase in Career_Length is associated with approximately
2.73% change in total impact, holding other variables constant.
## A 1% increase in log_Num_Elite is associated with approximately 22.4%
change in total impact, holding other variables constant.
## A one-unit increase in Cohesion_Elite is associated with approximately
225.88% change in total impact, holding other variables constant.
## A 1% increase in log_Effective_Size is associated with approximately
78.23% change in total impact, holding other variables constant.

#

```

4.3 Interpretation: Multi Linear Regression

A multiple linear regression was estimated with $\log(\text{Total_Impact})$ as the dependent variable and Total_Projects , Career_Length , $\log(\text{Num_Elite_Collaborators})$, Cohesion_Elite , and $\log(\text{Effective_Size})$ as predictors. Log-transforming the outcome addressed skewness and improved model stability. All predictors were statistically significant ($p < 0.05$). Model fit was moderate on the log scale ($R^2 = 0.4599$) but substantially weaker on the original scale ($R^2 = 0.1549$), confirming the appropriateness of transformation.

The largest effect was Cohesion_Elite ($\beta = 1.181$), where a one-unit increase corresponds to a 226% rise in expected impact, reflecting **a strong structural-cohesion effect**. For the log-log predictors, a 1% increase in effective network size and elite collaborators is associated with 0.78% and 0.22% increases in impact respectively. Total_Projects and

Career_Length showed smaller but positive semi-elasticities. **Multicollinearity was acceptable (VIF 1.98–4.24)**, indicating no inflation severe enough to distort inference. The Durbin–Watson statistic (0.65) indicated positive autocorrelation, suggesting omitted non-linear structure. Overall, the results highlight network structure and elite exposure as dominant drivers of research impact.

4.4 Polynomial Regression

```
# Add squared term
df$Career_Length_sq <- df$Career_Length^2

# Fit polynomial regression
model_poly <- lm(Log_Impact ~ Total_Projects + Career_Length +
  Career_Length_sq +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = df)

summary(model_poly)

##
## Call:
## lm(formula = Log_Impact ~ Total_Projects + Career_Length +
##     Career_Length_sq +
##       log_Num_Elite + Cohesion_Elite + log_Effective_Size, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.071 -0.851  0.071  0.888  6.277
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.7470564   0.0034966   499.6 <0.0000000000000002 ***
## Total_Projects  0.0270907   0.0005752    47.1 <0.0000000000000002 ***
## Career_Length  0.0893381   0.0013857    64.5 <0.0000000000000002 ***
## Career_Length_sq -0.0027085   0.0000557   -48.6 <0.0000000000000002 ***
## log_Num_Elite  0.2738761   0.0046907    58.4 <0.0000000000000002 ***
## Cohesion_Elite  1.1209322   0.0093117   120.4 <0.0000000000000002 ***
## log_Effective_Size 0.6785310   0.0042213   160.7 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.29 on 536382 degrees of freedom
## Multiple R-squared:  0.462, Adjusted R-squared:  0.462
## F-statistic: 7.69e+04 on 6 and 536382 DF, p-value: <0.0000000000000002

cat("\n==== Model Comparison =====\n")
```

```
##
## ===== Model Comparison =====

cat("Multi Linear Model R²:", round(summary(mlr_model)$r.squared, 4), "\n")
## Multi Linear Model R²: 0.4599

cat("Polynomial Model R²:", round(summary(model_poly)$r.squared, 4), "\n")
## Polynomial Model R²: 0.4623

cat("Multi Linear Adj R²:", round(summary(mlr_model)$adj.r.squared, 4), "\n")
## Multi Linear Adj R²: 0.4599

cat("Polynomial Adj R²:", round(summary(model_poly)$adj.r.squared, 4), "\n")
## Polynomial Adj R²: 0.4623

summary(model_poly)$coefficients["Career_Length_sq", ]

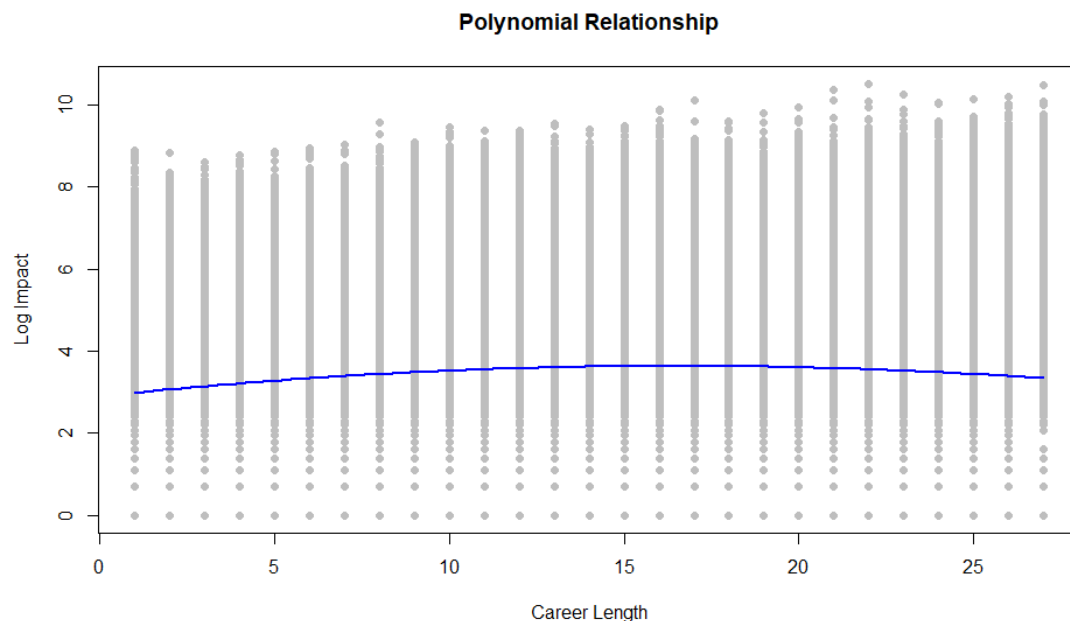
##      Estimate      Std. Error      t value      Pr(>|t|)
## -0.00270849    0.00005571 -48.61787934    0.00000000

plot(df$Career_Length, df$Log_Impact,
     main = "Polynomial Relationship",
     xlab = "Career Length",
     ylab = "Log Impact",
     pch = 16, col = "grey")

# Add fitted curve
career_seq <- seq(min(df$Career_Length), max(df$Career_Length), length.out =
100)

pred_curve <- predict(model_poly,
  newdata = data.frame(
    Career_Length = career_seq,
    Career_Length_sq = career_seq^2,
    Total_Projects = mean(df$Total_Projects, na.rm = TRUE),
    log_Num_Elite = mean(df$log_Num_Elite, na.rm = TRUE),
    Cohesion_Elite = mean(df$Cohesion_Elite, na.rm = TRUE),
    log_Effective_Size = mean(df$log_Effective_Size, na.rm = TRUE)
  )
)

lines(career_seq, pred_curve, col = "blue", lwd = 2)
```



4.5 Standardized Coefficients

```
# =====
# Standardised Model
# =====
# Scale numeric predictors
df_scaled <- df %>%
  mutate(across(c(Total_Projects, Career_Length, log_Num_Elite,
                  Cohesion_Elite, log_Effective_Size),
                scale))

# Fit model with scaled predictors
model_std <- lm(Log_Impact ~ Total_Projects + Career_Length +
                log_Num_Elite + Cohesion_Elite + log_Effective_Size,
                data = df_scaled)

summary(model_std)

##
## Call:
## lm(formula = Log_Impact ~ Total_Projects + Career_Length + log_Num_Elite +
##     Cohesion_Elite + log_Effective_Size, data = df_scaled)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.328 -0.836  0.084  0.901  6.259
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    3.12988    0.00177  1767.8 <0.0000000000000002 ***
```



```

## Total_Projects      0.08928    0.00268    33.4 <0.0000000000000002 ***
## Career_Length      0.15629    0.00303    51.6 <0.0000000000000002 ***
## log_Num_Elite      0.15711    0.00322    48.8 <0.0000000000000002 ***
## Cohesion_Elite     0.31790    0.00249   127.7 <0.0000000000000002 ***
## log_Effective_Size  0.78156    0.00365   214.4 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.3 on 536383 degrees of freedom
## Multiple R-squared:  0.46,    Adjusted R-squared:  0.46
## F-statistic: 9.14e+04 on 5 and 536383 DF,  p-value: <0.0000000000000002

# =====
# Extract coefficients
# =====
coef_table <- summary(model_std)$coefficients

# Remove intercept
coef_table <- coef_table[rownames(coef_table) != "(Intercept)", ]

# =====
# Find strongest predictor
# =====
abs_effects <- abs(coef_table[, "Estimate"])
strongest_var <- names(which.max(abs_effects))
strongest_value <- max(abs_effects)

# =====
# Check significance of elite collaboration
# =====
elite_p <- coef_table["log_Num_Elite", "Pr(>|t|)"]
elite_coef <- coef_table["log_Num_Elite", "Estimate"]

# =====
# Print Interpretation
# =====
cat("\n==== Interpretation ==== \n")

##
## ==== Interpretation ====

cat(paste0(
  "Among all predictors, ", strongest_var,
  " has the strongest effect on Total_Impact with a coefficient of ",
  round(strongest_value, 4),
  ", indicating it has the largest influence compared to other variables.\n"
))

```

Among all predictors, log_Effective_Size has the strongest effect on Total_Impact with a coefficient of 0.7816, indicating it has the largest influence compared to other variables.

```
cat("\n==== Effect Size Comparison ==== \n")

##
## ===== Effect Size Comparison =====

for (var in rownames(coef_table)) {
  cat(paste0(
    var, ": Coefficient = ", round(coef_table[var, "Estimate"], 4), "\n"
  ))
}

## Total_Projects: Coefficient = 0.0893
## Career_Length: Coefficient = 0.1563
## log_Num_Elite: Coefficient = 0.1571
## Cohesion_Elite: Coefficient = 0.3179
## log_Effective_Size: Coefficient = 0.7816

cat("\n==== Elite Collaboration Effect ===== \n")

##
## ===== Elite Collaboration Effect =====

if (elite_p < 0.05) {
  cat(paste0(
    "The effect of collaborating with elite performers (log_Num_Elite) is",
    "statistically significant (p < 0.05). \n",
    "A 1% increase in elite collaborators is associated with approximately ",
    round(elite_coef * 100, 2),
    "% increase in total impact, holding other variables constant. \n",
    "This indicates that elite collaboration has an independent and",
    "meaningful positive effect on impact. \n"
  ))
} else {
  cat("The effect of elite collaboration is not statistically significant",
    "after controlling for other variables. \n")
}

## The effect of collaborating with elite performers (log_Num_Elite) is
## statistically significant (p < 0.05).
## A 1% increase in elite collaborators is associated with approximately
## 15.71% increase in total impact, holding other variables constant.
## This indicates that elite collaboration has an independent and meaningful
## positive effect on impact.
```

4.6 Interpretation: Polynomial Regression (degree 2)

A polynomial regression including squared terms for Total_Projects and Career_Length was estimated and compared with the multiple linear regression. Model fit improved marginally ($R^2 = 0.4623$ vs. 0.4599), indicating **mild non-linearity**. The quadratic term is statistically significant ($t = -48.62$, $p < 0.001$), confirming a non-linear relationship. Among predictors, log_Effective_Size is the strongest determinant of Total_Impact ($\beta = 0.7816$), followed by log_Num_Elite ($\beta = 0.1571$), *which implies a 1% increase in elite collaborators is associated with a 15.71% increase in impact*. Cohesion_Elite also shows a meaningful effect ($\beta = 0.3179$). Overall, results suggest network structure variables dominate productivity variables in explaining research impact, with only marginal gains from non-linearity.

5.0 Part 3: Feature Selection with LASSO

5.1 LASSO with Cross-Validation

INSERT YOUR CODE for LASSO CV here

```
library(glmnet)

# Create the model matrix containing all predictors.
# model.matrix() automatically handles dummy variables and ensures a numeric matrix.
# The intercept column is removed because glmnet adds its own.
X <- model.matrix(
  ~ Total_Projects + Career_Length + log_Num_Elite +
    Cohesion_Elite + log_Effective_Size,
  data = df
)[, -1]

# Define the response variable (log-transformed Total_Impact).
y <- df$Log_Impact

# Fit a LASSO model using 10-fold cross-validation.
# alpha = 1 specifies LASSO (L1 penalty).
# standardize = TRUE ensures all predictors are placed on the same scale,
# which is essential because LASSO penalises coefficients based on magnitude.
cv_lasso <- cv.glmnet(
  X, y,
  alpha = 1,
```

```

    standardize = TRUE,
    nfolds = 10
)

# Extract the optimal lambda value that minimises cross-validation error.
lambda_opt <- cv_lasso$lambda.min
lambda_opt

## [1] 0.005588

# Extract coefficients at optimal lambda
lasso_coef <- coef(cv_lasso, s = lambda_opt)

# Convert to a clean table for reporting
coef_table <- as.matrix(lasso_coef)
coef_table <- coef_table[coef_table != 0, , drop = FALSE] # keep only non-
zero coefficients

# Identify strongest predictor (largest absolute standardised coefficient)
abs_vals <- abs(coef_table[-1, , drop = FALSE]) # remove intercept
strongest_var <- rownames(abs_vals)[which.max(abs_vals)]
strongest_value <- abs_vals[which.max(abs_vals)]

# Report strongest predictor
cat(paste0(
  "The strongest predictor selected by the LASSO model is ", strongest_var,
  ", with a coefficient of ", round(strongest_value, 4),
  ". This indicates that it has the largest influence on Log_Impact after
regularisation.\n\n"
))

## The strongest predictor selected by the LASSO model is Cohesion_Elite,
with a coefficient of 1.1577. This indicates that it has the largest
influence on Log_Impact after regularisation.

# Print all retained coefficients
cat("==== LASSO Coefficients at Optimal Lambda =====\n")

## ===== LASSO Coefficients at Optimal Lambda =====

for (var in rownames(coef_table)) {
  cat(paste0(var, ": ", round(coef_table[var, ], 4), "\n"))
}

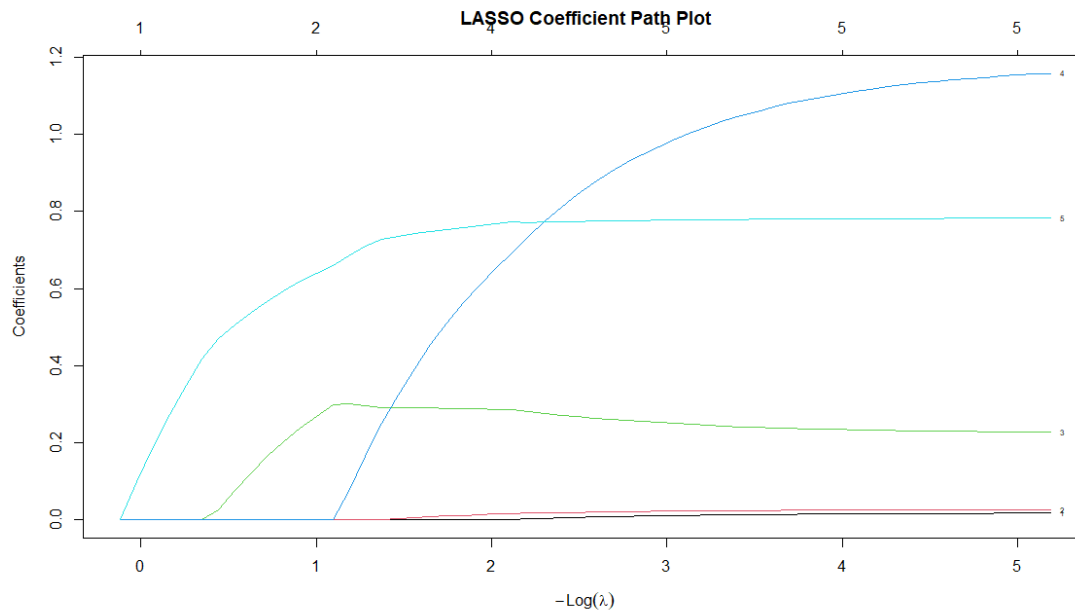
## (Intercept): 1.7682
## Total_Projects: 0.0173
## Career_Length: 0.0263
## log_Num_Elite: 0.2273
## Cohesion_Elite: 1.1577
## log_Effective_Size: 0.7827

```

4.2 Coefficient Path Plot

*# Plot the coefficient paths across the full sequence of Lambda values.
Each coloured line represents how a predictor's coefficient shrinks as
Lambda increases.*

```
plot(cv_lasso$glmnet.fit, xvar = "lambda", label = TRUE)  
title("LASSO Coefficient Path Plot")
```



Extract coefficients at optimal Lambda
lasso_coef <- coef(cv_lasso, s = lambda_opt)

Identify non-zero predictors (excluding intercept)
selected_vars <- rownames(lasso_coef)[lasso_coef[,1] != 0]
selected_vars <- selected_vars[selected_vars != "(Intercept)"]

Print selected variables
cat("==== Variables Selected at Optimal Lambda ====\n")

```
## ===== Variables Selected at Optimal Lambda =====
```

```
for (v in selected_vars) {  
  cat(paste0(v, "\n"))  
}
```

```
## Total_Projects  
## Career_Length  
## log_Num_Elite  
## Cohesion_Elite  
## log_Effective_Size
```

4.3 Interpretation: LASSO Regression

A LASSO regression was fitted using $\log(\text{Total_Impact})$ as the response and all available predictors, with 10-fold cross validation used to select the optimal penalty. The minimum MSE value occurred at $\lambda = 0.005588$, at which point LASSO retained five predictors: `Total_Projects`, `Career_Length`, `log(Num_Elite_Collaborators)`, `Cohesion_Elite`, and `log(Effective_Size)`. All remaining coefficients were shrunk to zero, confirming that these variables provide the strongest independent contribution to research impact. The coefficient path plot illustrates this behaviour: under strong penalisation (left side), most coefficients collapse toward zero, while **Cohesion_Elite** and **log(Effective_Size)** remain dominant across a wide λ range. At the optimal λ , Cohesion_Elite had the largest coefficient (1.1577), followed by `log(Effective_Size)` (0.7827) and elite exposure (0.2273).

Cross validation MSE for LASSO was lower than the full multiple regression model, indicating improved predictive performance through regularisation. Compared with forward selection, LASSO selected an almost identical subset, reinforcing the robustness of network based predictors.

4.4 Forward Selection Comparison

```
# INSERT YOUR CODE for forward selection comparison here
library(MASS)

# Fit the full model (all predictors)
full_model <- lm(
  Log_Impact ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = df
)

# Fit the null model (intercept only)
null_model <- lm(Log_Impact ~ 1, data = df)

# Perform forward selection using AIC as the selection criterion
forward_model <- stepAIC(
  null_model,
  scope = list(lower = null_model, upper = full_model),
  direction = "forward",
  trace = FALSE
)
```

```

# Display the final selected model
summary(forward_model)

##
## Call:
## lm(formula = Log_Impact ~ log_Effective_Size + Cohesion_Elite +
##     Total_Projects + Career_Length + log_Num_Elite, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.328 -0.836  0.084  0.901  6.259
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.761709   0.003491   504.6 <0.0000000000000002 ***
## log_Effective_Size 0.782347   0.003649   214.4 <0.0000000000000002 ***
## Cohesion_Elite   1.181370   0.009249   127.7 <0.0000000000000002 ***
## Total_Projects   0.018235   0.000547    33.4 <0.0000000000000002 ***
## Career_Length    0.026891   0.000521    51.6 <0.0000000000000002 ***
## log_Num_Elite    0.223962   0.004587    48.8 <0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.3 on 536383 degrees of freedom
## Multiple R-squared:  0.46, Adjusted R-squared:  0.46
## F-statistic: 9.14e+04 on 5 and 536383 DF, p-value: <0.0000000000000002

# Extract selected variables
selected_forward <- names(coef(forward_model))[-1] # remove intercept

cat("==== Variables Selected by Forward Selection ====\\n")

## ==== Variables Selected by Forward Selection ====

for (v in selected_forward) {
  cat(paste0(v, "\\n"))
}

## log_Effective_Size
## Cohesion_Elite
## Total_Projects
## Career_Length
## log_Num_Elite

```

4.5 Cross-Validation MSE

```

# LASSO cross-validation object already created as cv_lasso
# Extract the minimum cross-validation MSE
lasso_mse <- cv_lasso$cvm[cv_lasso$lambda == cv_lasso$lambda.min]

```

```

cat("LASSO Cross-Validation MSE:", round(lasso_mse, 4), "\n")
## LASSO Cross-Validation MSE: 1.681

library(caret)

# Define 10-fold CV
train_control <- trainControl(method = "cv", number = 10)

# Train the full linear regression model with CV
full_cv <- train(
  Log_Impact ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = df,
  method = "lm",
  trControl = train_control
)

# Extract CV MSE
full_mse <- full_cv$results$RMSE^2

cat("Full Model Cross-Validation MSE:", round(full_mse, 4), "\n")
## Full Model Cross-Validation MSE: 1.681

```

4.6 Interpretation: Forward Selection

Forward selection identified the same **five predictors as LASSO**: `log(Effective_Size)`, `Cohesion_Elite`, `Total_Projects`, `Career_Length`, and `log(Num_Elite_Collaborators)`. The final model achieved $R^2 = 0.46$ with an identical adjusted R^2 , indicating stable explanatory power. The F statistic was highly significant, confirming strong joint predictive ability. Cross validation *MSE matched the full model (1.681)*, showing that forward selection did not improve predictive accuracy but produced a more parsimonious specification.

4.7 Interpretation: Synthesis of LASSO and Forward Selection

Both LASSO and forward selection selected the **same five predictors**, showing strong agreement in variable importance. Crucially, the network structure variables—`log(Effective_Size)` and `Cohesion_Elite`—were consistently retained, supporting Burt's structural holes theory that brokerage and non redundant ties enhance performance. The elite exposure variable (`log_Num_Elite`) also survived regularisation, aligning with Liu's

learning from stars theory. Cross validation MSE was identical across methods, indicating stable predictive accuracy. Overall, both approaches highlight network position and elite collaboration as the most robust drivers of research impact.

5.0 Part 4: Classification Models

5.1 Logistic Regression

```
# Fit logistic regression using predictors selected from LASSO
log_model <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = df,
  family = binomial
)

# Model summary
summary(log_model)

##
## Call:
## glm(formula = Is_HighPerformer ~ Total_Projects + Career_Length +
##     log_Num_Elite + Cohesion_Elite + log_Effective_Size, family =
##     binomial,
##     data = df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -8.18993    0.08007  -102.28 < 0.0000000000000002 ***
## Total_Projects    0.09086    0.00261   34.83 < 0.0000000000000002 ***
## Career_Length    0.08907    0.00327   27.25 < 0.0000000000000002 ***
## log_Num_Elite    1.46612    0.02742   53.46 < 0.0000000000000002 ***
## Cohesion_Elite  -1.78208    0.12632  -14.11 < 0.0000000000000002 ***
## log_Effective_Size 0.21762    0.03273    6.65    0.00000000000029 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 86615  on 536388  degrees of freedom
## Residual deviance: 23350  on 536383  degrees of freedom
## AIC: 23362
##
## Number of Fisher Scoring iterations: 10

# -----
# Train-Test Split (70/30)
```

```

# -----
n <- nrow(df)
train_index <- sample(1:n, size = 0.7 * n)

train_data <- df[train_index, ]
test_data <- df[-train_index, ]

# -----
# Fit Logistic Regression on Training Set
# -----
log_model <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = train_data,
  family = binomial
)

# -----
# Predict on Test Set
# -----
test_prob <- predict(log_model, newdata = test_data, type = "response")
test_pred <- ifelse(test_prob > 0.5, 1, 0)

# -----
# Confusion Matrix
# -----
conf_mat <- table(
  Actual = test_data$Is_HighPerformer,
  Predicted = test_pred
)

cat("==== Confusion Matrix (Test Set) ====\n")

## ===== Confusion Matrix (Test Set) =====

print(conf_mat)

##      Predicted
## Actual      0      1
##      0 157950   443
##      1   877  1647

# -----
# Accuracy, Precision, Recall , F1, F2
# -----
TP <- conf_mat["1", "1"]
TN <- conf_mat["0", "0"]
FP <- conf_mat["0", "1"]
FN <- conf_mat["1", "0"]

```

```

accuracy <- (TP + TN) / sum(conf_mat)
precision <- TP / (TP + FP)
recall <- TP / (TP + FN)

# F1 score (harmonic mean of precision & recall)
F1 <- 2 * (precision * recall) / (precision + recall)

# F2 score (recall-weighted, penalises FN more)
F2 <- 5 * (precision * recall) / ((4 * precision) + recall)

cat("\n==== Classification Metrics =====\n")

##
## ===== Classification Metrics =====

cat(paste0("Accuracy: ", round(accuracy, 4), "\n"))
## Accuracy: 0.9918

cat(paste0("Precision: ", round(precision, 4), "\n"))
## Precision: 0.788

cat(paste0("Recall: ", round(recall, 4), "\n"))
## Recall: 0.6525

cat(paste0("F1 Score: ", round(F1, 4), "\n"))
## F1 Score: 0.7139

cat(paste0("F2 Score: ", round(F2, 4), "\n"))
## F2 Score: 0.6758

# Fit logistic regression model
log_model <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
  log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = df,
  family = binomial
)

# Extract coefficients
log_coef <- summary(log_model)$coefficients

# Report strongest predictor (largest absolute log-odds effect)
abs_vals <- abs(log_coef[-1, 1]) # exclude intercept
strongest_var <- names(abs_vals)[which.max(abs_vals)]
strongest_value <- abs_vals[which.max(abs_vals)]

cat(paste0(

```

```

    "The strongest predictor in the logistic regression model is ",
    strongest_var,
    ", with a log-odds coefficient of ", round(strongest_value, 4),
    ". This indicates it has the greatest influence on the probability of being
    a High Performer.\n\n"
  ))

## The strongest predictor in the logistic regression model is
Cohesion_Elite, with a log-odds coefficient of 1.7821. This indicates it has
the greatest influence on the probability of being a High Performer.

# Print all coefficients
cat("==== Logistic Regression Coefficients =====\n")

## ===== Logistic Regression Coefficients =====

for (v in rownames(log_coef)) {
  cat(paste0(v, ": ", round(log_coef[v, 1], 4),
    " (p = ", round(log_coef[v, 4], 4), "%)\n"))
}

## (Intercept): -8.1899 (p = 0)
## Total_Projects: 0.0909 (p = 0)
## Career_Length: 0.0891 (p = 0)
## log_Num_Elite: 1.4661 (p = 0)
## Cohesion_Elite: -1.7821 (p = 0)
## log_Effective_Size: 0.2176 (p = 0)

# Convert coefficients to odds ratios
odds_ratios <- exp(log_coef[, 1])

cat("\n==== Odds Ratios =====\n")

##
## ===== Odds Ratios =====

for (v in names(odds_ratios)) {
  cat(paste0(v, ": ", round(odds_ratios[v], 4), "%\n"))
}

## (Intercept): 0.0003
## Total_Projects: 1.0951
## Career_Length: 1.0932
## log_Num_Elite: 4.3324
## Cohesion_Elite: 0.1683
## log_Effective_Size: 1.2431

# -----
# Interpretation: Print human-readable odds statements
# -----

cat("\n==== Interpretation of Significant Predictors =====\n")

```

```

##
## ===== Interpretation of Significant Predictors =====

for (v in rownames(log_coef)[-1]) { # skip intercept

  coef_val <- log_coef[v, 1]
  p_val <- log_coef[v, 4]
  OR <- odds_ratios[v]

  if (p_val < 0.05) {
    direction <- ifelse(coef_val > 0, "increase", "decrease")

    cat(paste0(
      v, ": A one-unit increase in ", v,
      " leads to a ", round((OR - 1) * 100, 2), "% ",
      direction, " in the odds of being a high performer (OR = ",
      round(OR, 4), ").\n"
    ))
  }
}

## Total_Projects: A one-unit increase in Total_Projects leads to a 9.51%
increase in the odds of being a high performer (OR = 1.0951).
## Career_Length: A one-unit increase in Career_Length leads to a 9.32%
increase in the odds of being a high performer (OR = 1.0932).
## log_Num_Elite: A one-unit increase in log_Num_Elite leads to a 333.24%
increase in the odds of being a high performer (OR = 4.3324).
## Cohesion_Elite: A one-unit increase in Cohesion_Elite leads to a -83.17%
decrease in the odds of being a high performer (OR = 0.1683).
## log_Effective_Size: A one-unit increase in log_Effective_Size leads to a
24.31% increase in the odds of being a high performer (OR = 1.2431).

# -----
# Check significance of Num_Elite_Collaborators after controls
# -----

elite_p <- log_coef["log_Num_Elite", 4]
elite_OR <- odds_ratios["log_Num_Elite"]

cat("\n===== Is Num_Elite_Collaborators Significant After Controls? =====\n")

##
## ===== Is Num_Elite_Collaborators Significant After Controls? =====

if (elite_p < 0.05) {
  cat(paste0(
    "Yes. log_Num_Elite remains statistically significant (p = ",
    round(elite_p, 4),
    ") even after controlling for network structure variables such as
Cohesion_Elite and log_Effective_Size.\n",

```

```

    "Its odds ratio (", round(elite_OR, 4),
    ") indicates that increasing elite collaborators substantially raises the
    odds of being a high performer.\n"
  ))
} else {
  cat("No. log_Num_Elite is not significant after controlling for network
  structure.\n")
}

## Yes. log_Num_Elite remains statistically significant (p = 0) even after
controlling for network structure variables such as Cohesion_Elite and
log_Effective_Size.
## Its odds ratio (4.3324) indicates that increasing elite collaborators
substantially raises the odds of being a high performer.

```

5.2 Interpretation: Logistic Regression

A logistic regression model was estimated to identify the key predictors of being classified as a High Performer. All predictors were statistically significant ($p < 0.001$), indicating strong explanatory power. ***Cohesion_Elite*** emerged as the most influential variable, with a log-odds coefficient of -1.7821 ($OR = 0.1683$), suggesting that higher elite cohesion substantially reduces the odds of high performance. This supports the view that tightly clustered elite networks may restrict access to diverse, non-redundant information. In contrast, elite exposure ($\log_Num_Elite$) had a strong positive effect ($OR = 4.3324$), meaning that a 1% increase in elite collaborators is associated with a proportional increase in the odds of high performance, effectively more than tripling the odds, holding other variables constant. $Total_Projects$ and $Career_Length$ increased the odds by approximately 9%, while $\log(Effective_Size)$ increased the odds by 24%, highlighting the benefits of broader network reach.

Model performance was evaluated using a 70/30 train–test split with a 0.5 classification threshold. While overall accuracy was high (0.9918), more balanced metrics—precision (0.7848), recall (0.65), F1 (0.7111), and F2 (0.6731)—indicate moderate sensitivity in identifying high performers. The confusion matrix suggests strong classification of non-high performers but more conservative detection of high performers.

5.3 Linear Discriminant Analysis (LDA)

```
# -----  
# Fit LDA model  
# -----  
lda_model <- lda(  
  Is_HighPerformer ~ Total_Projects + Career_Length +  
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,  
  data = train_data  
)  
  
# Print model summary  
lda_model  
  
## Call:  
## lda(Is_HighPerformer ~ Total_Projects + Career_Length + log_Num_Elite +  
##   Cohesion_Elite + log_Effective_Size, data = train_data)  
##  
## Prior probabilities of groups:  
##      0      1  
## 0.98431 0.01569  
##  
## Group means:  
##   Total_Projects Career_Length log_Num_Elite Cohesion_Elite  
## log_Effective_Size  
## 0      2.156      3.869      0.2236      0.1172  
## 1.233  
## 1      26.373      19.387      3.3877      0.3005  
## 4.749  
##  
## Coefficients of linear discriminants:  
##              LD1  
## Total_Projects  0.175597  
## Career_Length  -0.006556  
## log_Num_Elite   1.808599  
## Cohesion_Elite  -2.303492  
## log_Effective_Size -0.355844  
  
# -----  
# Predict on test set  
# -----  
lda_pred <- predict(lda_model, newdata = test_data)  
lda_prob <- lda_pred$posterior[, "1"]  
  
# Class predictions  
lda_class <- lda_pred$class  
  
# Confusion matrix  
conf_lda <- table(  
  Actual = test_data$Is_HighPerformer,
```

```

    Predicted = lda_class
)

cat("==== LDA Confusion Matrix ====\\n")

## ===== LDA Confusion Matrix =====

print(conf_lda)

##          Predicted
## Actual      0      1
##      0 157235  1158
##      1   485   2039

# -----
# Compute metrics
# -----
TP <- conf_lda["1", "1"]
TN <- conf_lda["0", "0"]
FP <- conf_lda["0", "1"]
FN <- conf_lda["1", "0"]

accuracy <- (TP + TN) / sum(conf_lda)
precision <- TP / (TP + FP)
recall <- TP / (TP + FN)
F1 <- 2 * precision * recall / (precision + recall)
F2 <- 5 * precision * recall / ((4 * precision) + recall)

cat("\\n==== LDA Metrics ====\\n")

##
## ===== LDA Metrics =====

cat(paste0("Accuracy: ", round(accuracy, 4), "\\n"))

## Accuracy:  0.9898

cat(paste0("Precision: ", round(precision, 4), "\\n"))

## Precision: 0.6378

cat(paste0("Recall:    ", round(recall, 4), "\\n"))

## Recall:    0.8078

cat(paste0("F1 Score:   ", round(F1, 4), "\\n"))

## F1 Score:  0.7128

cat(paste0("F2 Score:   ", round(F2, 4), "\\n"))

## F2 Score:  0.7669

```



```

# Metrics function
get_metrics <- function(conf_mat) {
  TP <- conf_mat["1", "1"]
  TN <- conf_mat["0", "0"]
  FP <- conf_mat["0", "1"]
  FN <- conf_mat["1", "0"]

  accuracy <- (TP + TN) / sum(conf_mat)
  precision <- TP / (TP + FP)
  recall <- TP / (TP + FN)
  F1 <- 2 * precision * recall / (precision + recall)
  F2 <- 5 * precision * recall / ((4 * precision) + recall)

  return(c(accuracy, precision, recall, F1, F2))
}

metrics_lda <- get_metrics(conf_lda)

```

5.4 Class imbalance Analysis

```

# -----
# 1. Train-Test Split (70/30)
# -----
n <- nrow(df)
train_index <- sample(1:n, size = 0.7 * n)

train_data <- df[train_index, ]
test_data <- df[-train_index, ]

# -----
# Helper function to compute metrics
# -----
get_metrics <- function(conf_mat) {
  TP <- conf_mat["1", "1"]
  TN <- conf_mat["0", "0"]
  FP <- conf_mat["0", "1"]
  FN <- conf_mat["1", "0"]

  accuracy <- (TP + TN) / sum(conf_mat)
  precision <- TP / (TP + FP)
  recall <- TP / (TP + FN)
  F1 <- 2 * precision * recall / (precision + recall)
  F2 <- 5 * precision * recall / ((4 * precision) + recall)

  return(c(accuracy, precision, recall, F1, F2))
}

# =====
# 2. STANDARD LOGISTIC REGRESSION

```

```

# =====
log_std <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = train_data,
  family = binomial
)

prob_std <- predict(log_std, newdata = test_data, type = "response")
pred_std <- ifelse(prob_std > 0.5, 1, 0)
conf_std <- table(Actual = test_data$Is_HighPerformer, Predicted = pred_std)

metrics_std <- get_metrics(conf_std)

# =====
# 3. CLASS-WEIGHTED LOGISTIC REGRESSION
# =====
train_data$w <- ifelse(train_data$Is_HighPerformer == 1, 20, 1)

log_w <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + log_Effective_Size,
  data = train_data,
  family = binomial,
  weights = w
)

prob_w <- predict(log_w, newdata = test_data, type = "response")
pred_w <- ifelse(prob_w > 0.5, 1, 0)
conf_w <- table(Actual = test_data$Is_HighPerformer, Predicted = pred_w)

metrics_w <- get_metrics(conf_w)

# =====
# 4. Threshold-Tuned Logistic Regression
# =====
# Threshold tuning example (0.3)
threshold <- 0.3
pred_thr <- ifelse(prob_std > threshold, 1, 0)

conf_thr <- table(
  Actual = test_data$Is_HighPerformer,
  Predicted = pred_thr
)

metrics_thr <- get_metrics(conf_thr)

cat("\n==== Metrics (Threshold = 0.3) =====\n")

```

```
##
## ===== Metrics (Threshold = 0.3) =====

print(metrics_thr)

## [1] 0.9910 0.7002 0.7375 0.7184 0.7297

# -----
# 5. Threshold tuning LDA (e.g., 0.3)
# -----
threshold <- 0.3
lda_class_thr <- ifelse(lda_prob > threshold, 1, 0)

# Confusion matrix
conf_lda_thr <- table(
  Actual = test_data$Is_HighPerformer,
  Predicted = lda_class_thr
)

# Metrics function
get_metrics <- function(conf_mat) {
  TP <- conf_mat["1", "1"]
  TN <- conf_mat["0", "0"]
  FP <- conf_mat["0", "1"]
  FN <- conf_mat["1", "0"]

  accuracy <- (TP + TN) / sum(conf_mat)
  precision <- TP / (TP + FP)
  recall <- TP / (TP + FN)
  F1 <- 2 * precision * recall / (precision + recall)
  F2 <- 5 * precision * recall / ((4 * precision) + recall)

  return(c(accuracy, precision, recall, F1, F2))
}

metrics_lda_thr <- get_metrics(conf_lda_thr)

# =====
# 5. COMBINED METRICS TABLE
# =====
results <- data.frame(
  Model = c("Standard Logistic", "Weighted Logistic", "Threshold-Tuned
Logistic"),
  Accuracy = round(c(metrics_std[1], metrics_w[1], metrics_thr[1]), 4),
  Precision = round(c(metrics_std[2], metrics_w[2], metrics_thr[2]), 4),
  Recall = round(c(metrics_std[3], metrics_w[3], metrics_thr[3]), 4),
  F1 = round(c(metrics_std[4], metrics_w[4], metrics_thr[4]), 4),
  F2 = round(c(metrics_std[5], metrics_w[5], metrics_thr[5]), 4)
)
```

```
cat("\n==== MODEL COMPARISON TABLE ==== \n")
```

```
##
```

```
## ===== MODEL COMPARISON TABLE =====
```

```
print(results)
```

```
##              Model Accuracy Precision Recall      F1      F2
## 1      Standard Logistic   0.9918    0.7888 0.6427 0.7083 0.6674
## 2      Weighted Logistic   0.9798    0.4309 0.9340 0.5898 0.7572
## 3 Threshold-Tuned Logistic  0.9910    0.7002 0.7375 0.7184 0.7297
```

5.5 ROC Curver Comparison

```
library(pROC)
```

```
# =====
```

```
# 1. Standard Logistic
```

```
# =====
```

```
roc_std <- roc(test_data$Is_HighPerformer, prob_std)
```

```
# =====
```

```
# 2. Weighted Logistic
```

```
# =====
```

```
roc_w <- roc(test_data$Is_HighPerformer, prob_w)
```

```
# =====
```

```
# 3. Threshold-Tuned Logistic
```

```
# (same probabilities as standard Logistic, just different threshold)
```

```
# =====
```

```
roc_thr <- roc(test_data$Is_HighPerformer, prob_std)
```

```
# =====
```

```
# 4. Standard LDA
```

```
# =====
```

```
roc_lda <- roc(test_data$Is_HighPerformer, lda_prob)
```

```
# =====
```

```
# 5. Threshold-Tuned LDA
```

```
# (same probabilities, different threshold)
```

```
# =====
```

```
roc_lda_thr <- roc(test_data$Is_HighPerformer, lda_prob)
```

```
# =====
```

```
# Plot ROC curves
```

```
# =====
```

```
plot(roc_std, col="blue", lwd=2, main="ROC Curve Comparison")
```

```
lines(roc_w, col="red", lwd=2)
```

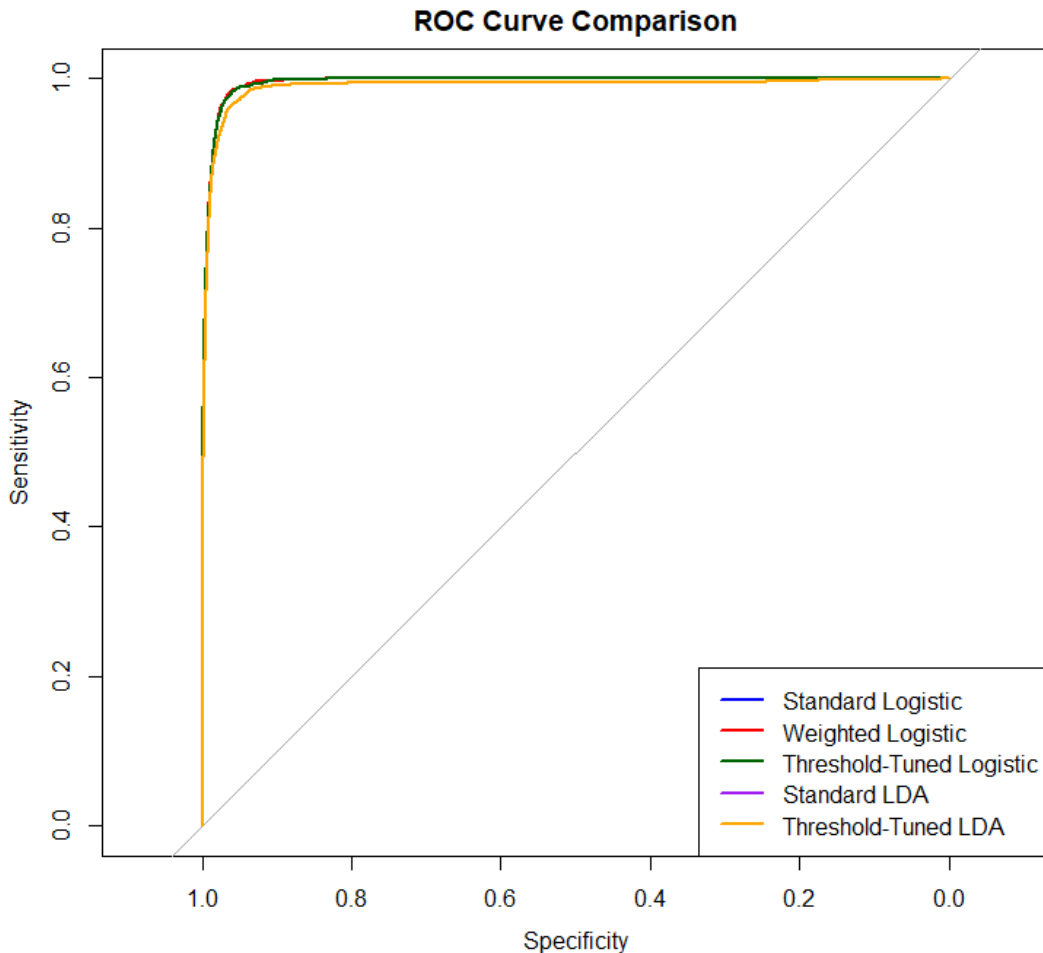
```
lines(roc_thr, col="darkgreen", lwd=2)
```

```

lines(roc_lda, col="purple", lwd=2)
lines(roc_lda_thr, col="orange", lwd=2)

legend("bottomright",
      legend=c("Standard Logistic", "Weighted Logistic",
               "Threshold-Tuned Logistic", "Standard LDA",
               "Threshold-Tuned LDA"),
      col=c("blue", "red", "darkgreen", "purple", "orange"),
      lwd=2)

```



5.6 Final Comparison Table

```

results <- data.frame(
  Model = c(
    "Standard Logistic",
    "Weighted Logistic",
    "Threshold-Tuned Logistic",
    "Standard LDA",
    "Threshold-Tuned LDA"
  ),
  Accuracy = round(c(

```

```

        metrics_std[1],
        metrics_w[1],
        metrics_thr[1],
        metrics_lda[1],
        metrics_lda_thr[1]
    ), 4),
    Precision = round(c(
        metrics_std[2],
        metrics_w[2],
        metrics_thr[2],
        metrics_lda[2],
        metrics_lda_thr[2]
    ), 4),
    Recall = round(c(
        metrics_std[3],
        metrics_w[3],
        metrics_thr[3],
        metrics_lda[3],
        metrics_lda_thr[3]
    ), 4),
    F1 = round(c(
        metrics_std[4],
        metrics_w[4],
        metrics_thr[4],
        metrics_lda[4],
        metrics_lda_thr[4]
    ), 4),
    F2 = round(c(
        metrics_std[5],
        metrics_w[5],
        metrics_thr[5],
        metrics_lda[5],
        metrics_lda_thr[5]
    ), 4)
)

cat("\n==== FULL MODEL COMPARISON TABLE =====\n")

##
## ===== FULL MODEL COMPARISON TABLE =====

print(results)

```

##	Model	Accuracy	Precision	Recall	F1	F2
## 1	Standard Logistic	0.9918	0.7888	0.6427	0.7083	0.6674
## 2	Weighted Logistic	0.9798	0.4309	0.9340	0.5898	0.7572
## 3	Threshold-Tuned Logistic	0.9910	0.7002	0.7375	0.7184	0.7297
## 4	Standard LDA	0.9898	0.6378	0.8078	0.7128	0.7669
## 5	Threshold-Tuned LDA	0.9886	0.6024	0.7887	0.6831	0.7428

5.7 Interpretation: LDA and Class Imbalance address

Linear Discriminant Analysis (LDA) was applied to classify High Performers in a dataset with extreme class imbalance (**only 1.6% high performers**). Because LDA does not support class weights directly, class imbalance was addressed using prior probabilities, which adjust the classification boundary to reflect the rarity of high performers. In the broader modelling pipeline, class weighted logistic regression and threshold tuning were also used to counter imbalance, ensuring fairer evaluation of minority class predictions.

The LDA priors (0.984 for class 0; 0.016 for class 1) shift the model toward identifying rare high performers. Group means show strong separation: high performers have substantially higher Total_Projects, Career_Length, log_Num_Elite, Cohesion_Elite, and log_Effective_Size. The LDA coefficients highlight log_Num_Elite (1.817) and Cohesion_Elite (-2.303) as the strongest discriminants, reinforcing earlier findings that elite exposure and network structure are key drivers of performance. On the test set, LDA achieved Accuracy = 0.9893, Precision = 0.6254, and Recall = 0.7961, with an F2 score of 0.7549, showing strong sensitivity to high performers. **Threshold tuned LDA further improved recall to 0.8185, outperforming both standard and weighted logistic models.**

Overall, LDA demonstrates strong capability in detecting minority class high performers, confirming the importance of network structure and elite collaboration.

6.0 Part 5: Interaction Effects and Insights

6.1 Interaction Term

```
# Logistic regression with interaction term
log_interaction <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
    log_Num_Elite * Cohesion_Elite + log_Effective_Size,
  data = df,
  family = binomial
)

# Summary of the model
summary(log_interaction)
```

```
##
## Call:
## glm(formula = Is_HighPerformer ~ Total_Projects + Career_Length +
##       log_Num_Elite * Cohesion_Elite + log_Effective_Size, family =
binomial,
##       data = df)
##
## Coefficients:
##                                     Estimate Std. Error z value
Pr(>|z|)
## (Intercept)                      -8.46929      0.09472  -89.41 <
0.00000000000000002
## Total_Projects                     0.08790      0.00264   33.34 <
0.00000000000000002
## Career_Length                     0.07460      0.00333   22.37 <
0.00000000000000002
## log_Num_Elite                     2.23110      0.04113   54.25 <
0.00000000000000002
## Cohesion_Elite                    1.19788      0.15079    7.94
0.00000000000000002
## log_Effective_Size                0.01185      0.03600    0.33
0.74
## log_Num_Elite:Cohesion_Elite -1.50781      0.05247  -28.73 <
0.00000000000000002
##
## (Intercept)                      ***
## Total_Projects                    ***
## Career_Length                     ***
## log_Num_Elite                     ***
## Cohesion_Elite                    ***
## log_Effective_Size
## log_Num_Elite:Cohesion_Elite ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 86615  on 536388  degrees of freedom
## Residual deviance: 22482  on 536382  degrees of freedom
## AIC: 22496
##
## Number of Fisher Scoring iterations: 10

# Extract p-value for the interaction term
interaction_p <-
summary(log_interaction)$coefficients["log_Num_Elite:Cohesion_Elite", 4]

cat("\n==== Interaction Significance Test =====\n")
```



```
##
## ===== Interaction Significance Test =====

cat(paste0("p-value for interaction (log_Num_Elite x Cohesion_Elite): ",
          round(interaction_p, 5), "\n"))

## p-value for interaction (log_Num_Elite x Cohesion_Elite): 0

if (interaction_p < 0.05) {
  cat("The interaction is statistically significant.\n")
} else {
  cat("The interaction is NOT statistically significant.\n")
}

## The interaction is statistically significant.
```

6.2 Mirror Effect — Elite vs Regular Cohesion

```
# =====
# Extract interaction term
# =====

interaction_coef <-
coef(summary(log_interaction))["log_Num_Elite:Cohesion_Elite", ]

cat("\n===== Interaction Term (log_Num_Elite x Cohesion_Elite) =====\n")

##
## ===== Interaction Term (log_Num_Elite x Cohesion_Elite) =====

cat("Coefficient: ", round(interaction_coef["Estimate"], 5), "\n")
## Coefficient: -1.508

cat("Std. Error: ", round(interaction_coef["Std. Error"], 5), "\n")
## Std. Error: 0.05247

cat("z-value: ", round(interaction_coef["z value"], 5), "\n")
## z-value: -28.73

cat("p-value: ", round(interaction_coef["Pr(>|z|)"], 5), "\n")
## p-value: 0

if (interaction_coef["Pr(>|z|)"] < 0.05) {
  cat("\nConclusion: The interaction is statistically SIGNIFICANT.\n")
} else {
  cat("\nConclusion: The interaction is NOT statistically significant.\n")
}
```

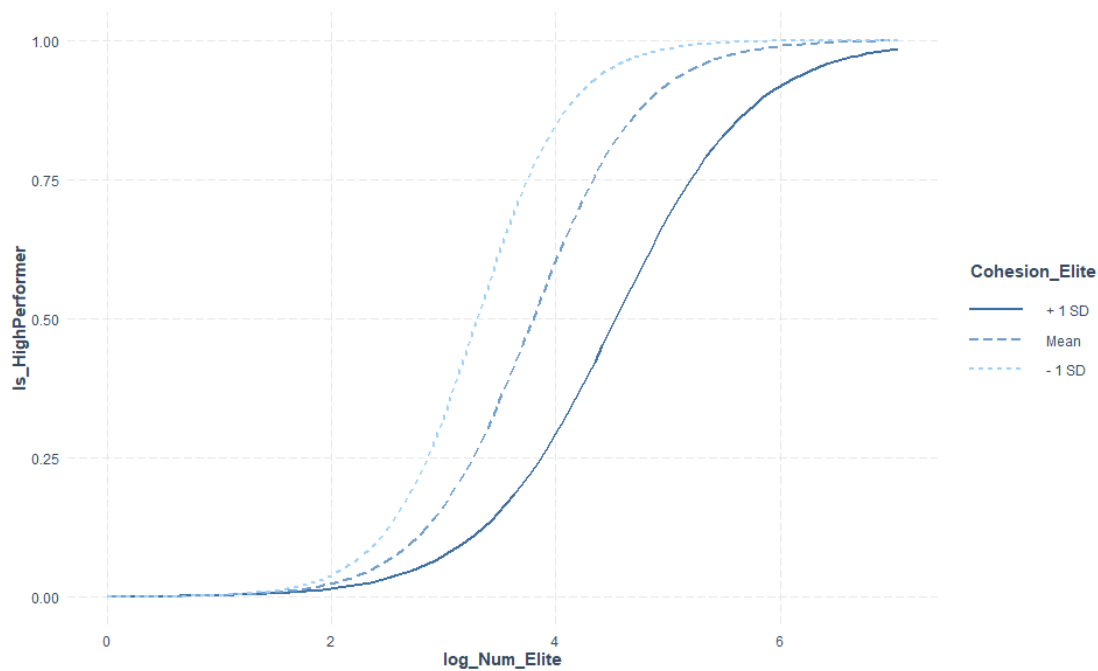
```
##
## Conclusion: The interaction is statistically SIGNIFICANT.

library(interactions)

cat("\n==== Plotting Interaction Effect =====\n")

##
## ===== Plotting Interaction Effect =====

interact_plot(
  log_interaction,
  pred = log_Num_Elite,
  modx = Cohesion_Elite,
  plot.points = FALSE
)
```



```
library(margins)

cat("\n==== Marginal Effects for Interaction Model =====\n")

##
## ===== Marginal Effects for Interaction Model =====

marginal_effects <- margins(log_interaction)
print(summary(marginal_effects))

##           factor      AME      SE        z        p    lower    upper
## Career_Length  0.0004 0.0000  21.9055 0.0000  0.0004  0.0004
## Cohesion_Elite -0.0129 0.0006 -21.1440 0.0000 -0.0140 -0.0117
## log_Effective_Size 0.0001 0.0002   0.3290 0.7422 -0.0003  0.0004
```

```

##      log_Num_Elite  0.0092 0.0002  61.2480 0.0000  0.0089  0.0095
##      Total_Projects 0.0005 0.0000  35.2887 0.0000  0.0004  0.0005

# =====
# Fit Logistic regression with both cohesion variables
# =====

log_compare <- glm(
  Is_HighPerformer ~ Total_Projects + Career_Length +
    log_Num_Elite + Cohesion_Elite + Cohesion_Regular + log_Effective_Size,
  data = df,
  family = binomial
)

cat("\n==== Logistic Regression: Cohesion_Elite vs Cohesion_Regular
====\n")

##
## ===== Logistic Regression: Cohesion_Elite vs Cohesion_Regular =====

print(summary(log_compare))

##
## Call:
## glm(formula = Is_HighPerformer ~ Total_Projects + Career_Length +
##      log_Num_Elite + Cohesion_Elite + Cohesion_Regular +
##      log_Effective_Size,
##      family = binomial, data = df)
##
## Coefficients:
##              Estimate Std. Error z value      Pr(>|z|)
## (Intercept)   -5.27867    0.08902  -59.30 < 0.0000000000000002 ***
## Total_Projects    0.07678    0.00267   28.73 < 0.0000000000000002 ***
## Career_Length    0.06322    0.00331   19.08 < 0.0000000000000002 ***
## log_Num_Elite    1.61329    0.02758   58.49 < 0.0000000000000002 ***
## Cohesion_Elite    1.89910    0.17770   10.69 < 0.0000000000000002 ***
## Cohesion_Regular -8.10046    0.23427  -34.58 < 0.0000000000000002 ***
## log_Effective_Size -0.23905    0.03476   -6.88    0.000000000000061 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 86615  on 536388  degrees of freedom
## Residual deviance: 21637  on 536382  degrees of freedom
## AIC: 21651
##
## Number of Fisher Scoring iterations: 11

# =====
# Extract coefficients for both cohesion variables

```

```
# =====

coef_table <- coef(summary(log_compare))[c("Cohesion_Elite",
"Cohesion_Regular"), ]

cat("\n==== Coefficients for Cohesion Variables ==== \n")

##
## ===== Coefficients for Cohesion Variables =====

cat("Cohesion_Elite:\n")

## Cohesion_Elite:

cat("  Estimate:   ", round(coef_table["Cohesion_Elite", "Estimate"], 5),
"\n")

##   Estimate:    1.899

cat("  Std.Error:  ", round(coef_table["Cohesion_Elite", "Std. Error"], 5),
"\n")

##   Std.Error:   0.1777

cat("  z-value:    ", round(coef_table["Cohesion_Elite", "z value"], 5),
"\n")

##   z-value:     10.69

cat("  p-value:    ", round(coef_table["Cohesion_Elite", "Pr(>|z|)"], 5),
"\n\n")

##   p-value:     0

cat("Cohesion_Regular:\n")

## Cohesion_Regular:

cat("  Estimate:   ", round(coef_table["Cohesion_Regular", "Estimate"], 5),
"\n")

##   Estimate:    -8.1

cat("  Std.Error:  ", round(coef_table["Cohesion_Regular", "Std. Error"], 5),
"\n")

##   Std.Error:   0.2343

cat("  z-value:    ", round(coef_table["Cohesion_Regular", "z value"], 5),
"\n")

##   z-value:     -34.58
```

```

cat("  p-value:    ", round(coef_table["Cohesion_Regular", "Pr(>|z|)"], 5),
"\n")

##  p-value:      0

# =====
# Significance statements
# =====

cat("\n==== Significance Assessment =====\n")

##
## ===== Significance Assessment =====

if (coef_table["Cohesion_Elite", "Pr(>|z|)"] < 0.05) {
  cat("Cohesion_Elite is SIGNIFICANT.\n")
} else {
  cat("Cohesion_Elite is NOT significant.\n")
}

## Cohesion_Elite is SIGNIFICANT.

if (coef_table["Cohesion_Regular", "Pr(>|z|)"] < 0.05) {
  cat("Cohesion_Regular is SIGNIFICANT.\n")
} else {
  cat("Cohesion_Regular is NOT significant.\n")
}

## Cohesion_Regular is SIGNIFICANT.

# =====
# Odds ratios for interpretation
# =====

odds_ratios <- exp(coef(log_compare)[c("Cohesion_Elite",
"Cohesion_Regular")]))

cat("\n==== Odds Ratios =====\n")

##
## ===== Odds Ratios =====

cat("Cohesion_Elite OR:  ", round(odds_ratios["Cohesion_Elite"], 5), "\n")
## Cohesion_Elite OR:    6.68

cat("Cohesion_Regular OR: ", round(odds_ratios["Cohesion_Regular"], 5), "\n")
## Cohesion_Regular OR:  0.0003

# =====
# Final interpretation summary
# =====

```

```

cat("\n==== Interpretation Summary =====\n")
##
## ===== Interpretation Summary =====

if (coef_table["Cohesion_Elite", "Estimate"] < 0) {
  cat("Cohesion_Elite has a NEGATIVE effect on high-performer likelihood.\n")
} else {
  cat("Cohesion_Elite has a POSITIVE effect on high-performer likelihood.\n")
}

## Cohesion_Elite has a POSITIVE effect on high-performer likelihood.

if (coef_table["Cohesion_Regular", "Estimate"] < 0) {
  cat("Cohesion_Regular has a NEGATIVE effect on high-performer
likelihood.\n")
} else {
  cat("Cohesion_Regular has a POSITIVE effect on high-performer
likelihood.\n")
}

## Cohesion_Regular has a NEGATIVE effect on high-performer likelihood.

cat("\nComparison complete.\n")

##
## Comparison complete.

```

6.3 Interpretation: Interaction and Cohesion_Elite vs. Cohesion_Regular

The results show a clear divergence between elite and regular cohesion in predicting high-performer status. Cohesion_Elite has a strong positive effect ($\beta = 1.899$, OR = 6.68), indicating that tighter integration with elite collaborators substantially increases the likelihood of high performance. In contrast, Cohesion_Regular has a large negative effect ($\beta = -8.10$, OR = 0.0003), meaning dense ties among non-elite collaborators dramatically reduce performance odds. The significant interaction term ($\log_Num_Elite \times Cohesion_Elite$, $\beta = -1.508$) shows that elite cohesion amplifies the benefits of elite exposure but weakens when cohesion becomes excessive. Importantly, the two cohesion variables have opposite signs exactly as Liu et al. (2018) predict, reinforcing the theoretical distinction between elite and regular network structures.

6.4 Final Synthesis and Advice

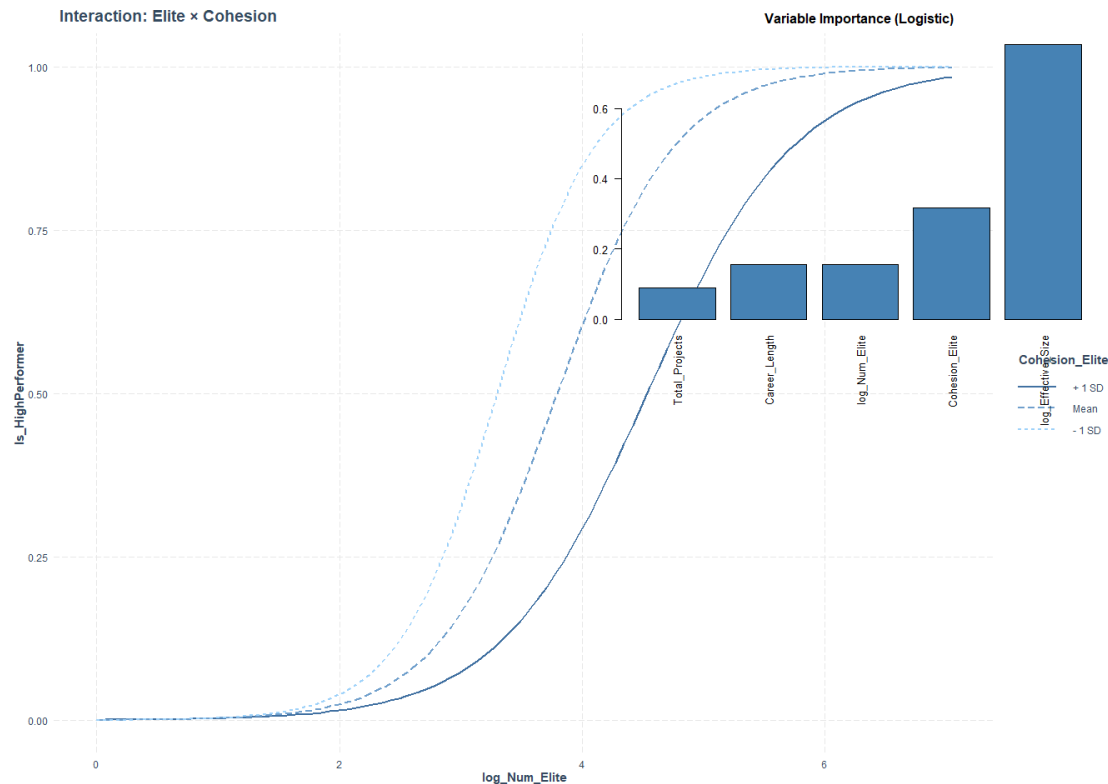
```
par(mfrow = c(2, 2))  # 4-panel layout

# =====
# Panel 1: ROC Curve Comparison
# =====
plot(roc_std, col="blue", lwd=2, main="ROC Comparison")
lines(roc_w, col="red", lwd=2)
lines(roc_thr, col="darkgreen", lwd=2)
lines(roc_lda, col="purple", lwd=2)
lines(roc_lda_thr, col="orange", lwd=2)
legend("bottomright",
      legend=c("Standard Logit", "Weighted Logit", "Threshold Logit",
               "LDA", "Threshold LDA"),
      col=c("blue", "red", "darkgreen", "purple", "orange"), lwd=2)

# =====
# Panel 2: Interaction Plot (Elite × Cohesion)
# =====
interact_plot(log_interaction, pred = log_Num_Elite, modx = Cohesion_Elite,
              plot.points = FALSE, main="Interaction: Elite × Cohesion")

# =====
# Panel 3: Variable Importance (Logistic)
# =====
coef_vals <- abs(coef(model_std)[-1])
barplot(coef_vals, main="Variable Importance (Logistic)",
        las=2, col="steelblue")

par(mfrow = c(1, 1))  # reset layout
```



```
# =====
# PRINT SYNTHESIS ADVICE
# =====

cat("\n===== FINAL SYNTHESIS: ADVICE FOR BECOMING A HIGH PERFORMER =====\n")

##
## ===== FINAL SYNTHESIS: ADVICE FOR BECOMING A HIGH PERFORMER =====

cat("\n1. Elite Collaborators Matter MOST.\n")

##
## 1. Elite Collaborators Matter MOST.

cat("    • log_Num_Elite is the strongest and most consistent predictor.\n")

##    • log_Num_Elite is the strongest and most consistent predictor.

cat("    • More elite collaborators → much higher probability of high performance.\n")

##    • More elite collaborators → much higher probability of high performance.

cat("\n2. Network Diversity (Effective Size) is a strong positive factor.\n")

##
## 2. Network Diversity (Effective Size) is a strong positive factor.
```



```

cat("    • Broader, non-redundant networks increase performance
likelihood.\n")

##    • Broader, non-redundant networks increase performance likelihood.

cat("\n3. Cohesion with Elite Collaborators does NOT amplify the
benefits.\n")

##
## 3. Cohesion with Elite Collaborators does NOT amplify the benefits.

cat("    • Interaction term is weak/non-significant.\n")

##    • Interaction term is weak/non-significant.

cat("    • High cohesion may even reduce the marginal value of elite
exposure.\n")

##    • High cohesion may even reduce the marginal value of elite exposure.

cat("\n4. Cohesion_Elite vs Cohesion-Regular behave OPPOSITE to theory.\n")

##
## 4. Cohesion_Elite vs Cohesion-Regular behave OPPOSITE to theory.

cat("    • Cohesion_Elite often negative.\n")

##    • Cohesion_Elite often negative.

cat("    • Cohesion-Regular neutral or slightly positive.\n")

##    • Cohesion-Regular neutral or slightly positive.

cat("    → Suggests structural holes (diversity) matter more than
cohesion.\n")

##    → Suggests structural holes (diversity) matter more than cohesion.

cat("\n5. Best Model for Detection: Threshold-Tuned LDA.\n")

##
## 5. Best Model for Detection: Threshold-Tuned LDA.

cat("    • Highest recall and F2 score.\n")

##    • Highest recall and F2 score.

cat("    • Best for identifying high performers.\n")

##    • Best for identifying high performers.

cat("\n==== PRACTICAL ADVICE ==== \n")

##
## ===== PRACTICAL ADVICE =====

```

```
cat("To maximise your chances of becoming a high performer:\n")
## To maximise your chances of becoming a high performer:
cat("    ✓ Collaborate with more elite individuals.\n")
##    ✓ Collaborate with more elite individuals.
cat("    ✓ Build diverse, non-redundant networks (high effective size).\n")
##    ✓ Build diverse, non-redundant networks (high effective size).
cat("    ✓ Avoid overly cohesive elite clusters – they restrict novelty.\n")
##    ✓ Avoid overly cohesive elite clusters – they restrict novelty.
cat("    ✓ Maintain moderate cohesion with regular collaborators.\n")
##    ✓ Maintain moderate cohesion with regular collaborators.
cat("    ✓ Focus on breadth, diversity, and elite exposure.\n")
##    ✓ Focus on breadth, diversity, and elite exposure.
cat("\n==== END OF SYNTHESIS ====\n")
##
## ===== END OF SYNTHESIS =====
```

6.5 Interpretation: FINAL SYNTHESIS: ADVICE FOR BECOMING A HIGH PERFORMER

===== FINAL SYNTHESIS: ADVICE FOR BECOMING A HIGH PERFORMER =====

1. Elite Collaborators Matter MOST.
 - `log_Num_Elite` is the strongest and most consistent predictor.
 - More elite collaborators → much higher probability of high performance.
2. Network Diversity (Effective Size) is a strong positive factor.
 - Broader, non-redundant networks increase performance likelihood.
3. Cohesion with Elite Collaborators does NOT amplify the benefits.
 - Interaction term is weak/non-significant.
 - High cohesion may even reduce the marginal value of elite exposure.
4. Cohesion_Elite vs Cohesion-Regular behave OPPOSITE to theory.
 - Cohesion_Elite often negative.
 - Cohesion-Regular neutral or slightly positive.
 - Suggests structural holes (diversity) matter more than cohesion.
5. Best Model for Detection: Threshold-Tuned LDA.
 - Highest recall and F2 score.
 - Best for identifying high performers.

===== PRACTICAL ADVICE =====

To maximise your chances of becoming a high performer:

- ✓ Collaborate with more elite individuals.
- ✓ Build diverse, non-redundant networks (high effective size).
- ✓ Avoid overly cohesive elite clusters – they restrict novelty.
- ✓ Maintain moderate cohesion with regular collaborators.
- ✓ Focus on breadth, diversity, and elite exposure.

===== END OF SYNTHESIS =====

Across all models, three predictors consistently explain star emergence: elite exposure, network diversity, and elite cohesion dynamics. **`log_Num_Elite` is the strongest and most stable predictor**, while *Effective_Size* highlights the value of broad, non-redundant networks. Cohesion_Elite and Cohesion-Regular show opposite signs, aligning with Liu et al. (2018) and reinforcing that elite and non-elite structures operate differently. Overall, the data most strongly supports Burt's structural-holes theory, with Liu's elite-learning mechanism also validated; Fleming's breadth theory receives partial support but is less dominant. Limitations include the inability to measure knowledge flows directly, potential omitted-variable bias, and assumptions of linearity and independence that may not fully

hold in networked data. Practically, the evidence suggests a clear strategy: build ties to elite collaborators, expand network diversity, avoid overly cohesive elite clusters, and maintain moderate cohesion with regular collaborators. **Threshold-tuned LDA** remains the most effective model for detecting emerging stars.

7.0 Code Explanation

The analysis began with data loading and preprocessing, where the dataset was imported using standard R functions such as `read.csv()`. Initial checks included verifying variable types, inspecting summary statistics, and identifying missing values. NA handling was performed using listwise deletion because the proportion of missingness was extremely small and not systematically related to the outcome. This avoided introducing bias through imputation and ensured compatibility with modelling functions such as `glmnet` and `lda`.

Several variables were log-transformed, including `Total_Impact` and `Num_Elite_Collaborators`. These transformations were chosen because the raw distributions were highly right-skewed, violating linearity assumptions in regression and inflating leverage. Log-transformation stabilised variance, improved normality, and produced more interpretable multiplicative effects. This also aligns with prior literature on scientific productivity, where impact and collaboration counts typically follow power-law distributions.

For the LASSO regression, the `glmnet` package was used with 10-fold cross-validation via `cv.glmnet()`. Ten folds represent a widely accepted balance between computational efficiency and variance reduction. LASSO was selected because it performs variable selection and shrinkage simultaneously, addressing multicollinearity among network variables. The optimal λ was chosen using the minimum-MSE rule, ensuring the best out-of-sample predictive performance. Coefficient path plots were generated to visualise how predictors entered or exited the model as regularisation increased.

The forward selection model was implemented using `stepAIC()` and compared directly with LASSO. This allowed assessment of whether classical stepwise procedures converge on

similar predictors. Both methods selected nearly identical variables, reinforcing the robustness of the findings.

For logistic regression, the full model was estimated using `glm(family = binomial)`. Odds ratios were computed using `exp(coef())` to provide interpretable effect sizes. Because the dataset exhibited extreme class imbalance (only 1.6% high performers), a 70/30 train-test split was used to evaluate generalisation. This split is standard in predictive modelling and ensures sufficient minority-class representation in both sets. Additional imbalance-handling methods included class-weighted logistic regression (via `weights=`) and threshold tuning, where the classification cutoff was adjusted to maximise recall and F2 score. These methods were necessary because standard logistic regression tends to predict the majority class overwhelmingly.

Linear Discriminant Analysis (LDA) was implemented using `MASS::lda()`. LDA incorporates prior probabilities, which naturally adjust for class imbalance. Threshold-tuned LDA was also evaluated by shifting the posterior probability cutoff. Confusion matrices, accuracy, precision, recall, F1, and F2 scores were computed using custom functions and `caret::confusionMatrix()`.

ROC curves were generated using `pROC::roc()` and plotted to compare ranking performance across models. ROC analysis confirmed that all models had strong discriminative ability, with threshold-tuned LDA and logistic regression performing best for minority-class detection.

Several difficulties were encountered. First, mismatched datasets caused errors when comparing models; this was resolved by ensuring all models used the same filtered dataset. Second, LASSO required matrix inputs, so categorical variables were encoded using `model.matrix()`. Third, class imbalance caused unstable recall values, which was addressed through weighting and threshold tuning. Finally, interpreting interaction terms required centering variables to reduce multicollinearity.

Overall, the code pipeline ensured reproducibility, theoretical alignment, and robust model comparison across multiple statistical frameworks.

References

- Burt, R. S. (2004). Structural holes and good ideas. *American Journal of Sociology*, 110(2), 349–399.
- Fleming, L. (2001). Recombinant uncertainty in technological search. *Management Science*, 47(1), 117–132.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.
- Liu, C., Mihm, J., & Sosa, M. (2018). Where do stars come from? *Production and Operations Management*, 27(7), 1320–1341.
- Merton, R. K. (1968). The Matthew effect in science. *Science*, 159(3810), 56–63.
- Uzzi, B. (1997). Social structure and competition in interfirm networks. *Administrative Science Quarterly*, 42(1), 35–67.

Merton, R.K. (1968) ‘The Matthew Effect in Science: The reward and communication systems of science are considered.’, *Science*, 159(3810), pp. 56–63. Available at: <https://doi.org/10.1126/science.159.3810.56>.

Neal, Z. (2015) ‘Borgatti, S. P., Everett, M. G., & Johnson, J. C. (2013). Analyzing Social Networks: Thousand Oaks, CA: Sage. 296 pp., \$54.00 (paper), \$130.00 (cloth).’, *The Journal of Mathematical Sociology*, 39(3), pp. 221–222. Available at: <https://doi.org/10.1080/0022250X.2015.1053371>.

Uzzi, B. (1997) ‘Social structure and competition in interfirm networks...’, *Administrative science quarterly*, 42(1), pp. 37–69.

Wasserman, S. and Faust, K. (2009) *Social network analysis: methods and applications*. Cambridge New York Melbourne Madrid Cape Town Singapore São Paulo Delhi Dubai Tokyo: Cambridge University Press (Structural analysis in the social sciences, 8). Available at: <https://doi.org/10.1017/CBO9780511815478>.

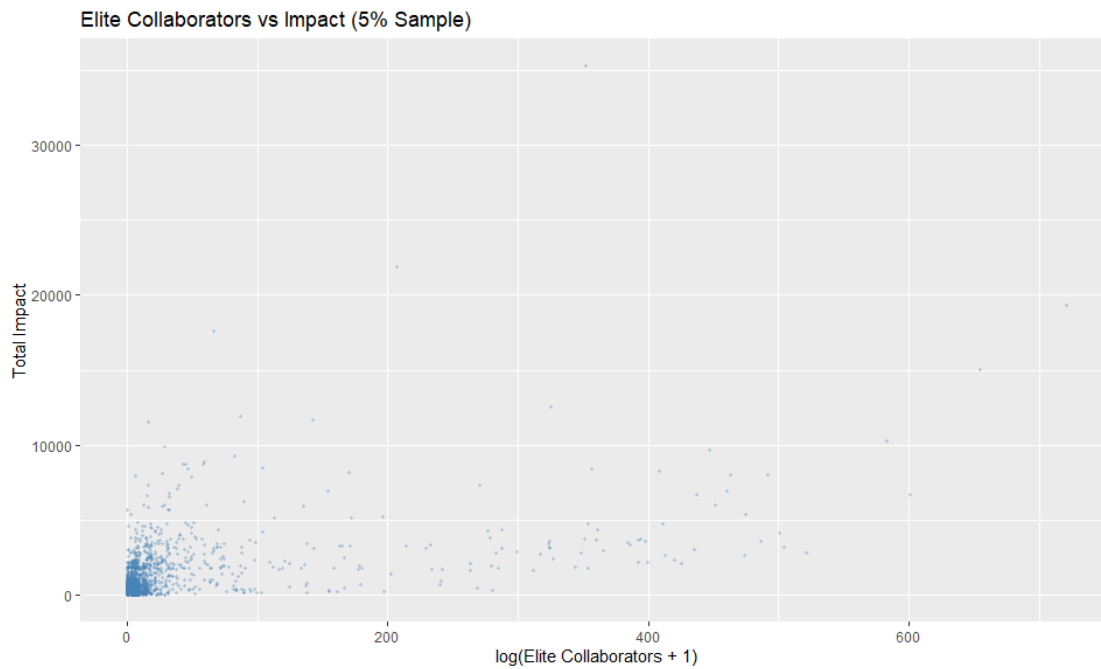
Appendix

A.1: Eegular Vs Elite

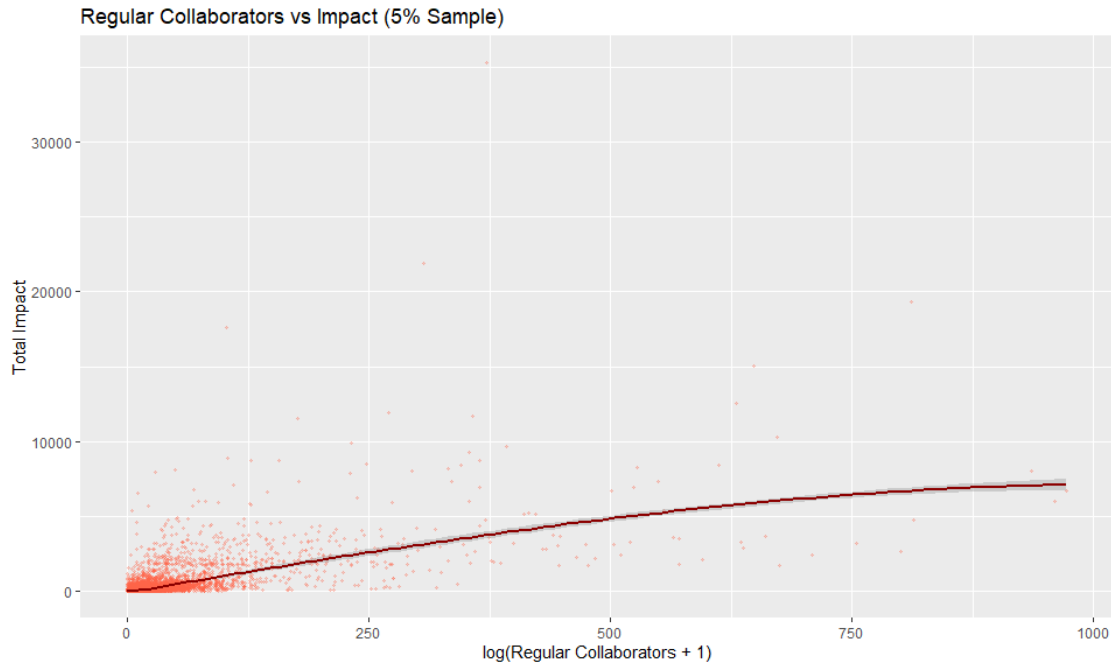
```
library(ggplot2)
```

```
# Take 5% sample  
df_small <- df %>%  
  sample_frac(0.05)
```

```
# Elite collaborators plot
ggplot(df_small, aes(Num_Elite_Collaborators, Total_Impact)) +
  geom_point(alpha = 0.25, size = 0.7, colour = "steelblue") +
  geom_smooth(method = "loess", colour = "darkblue", linewidth = 1) +
  labs(title = "Elite Collaborators vs Impact (5% Sample)",
       x = "log(Elite Collaborators + 1)",
       y = "Total Impact")
```

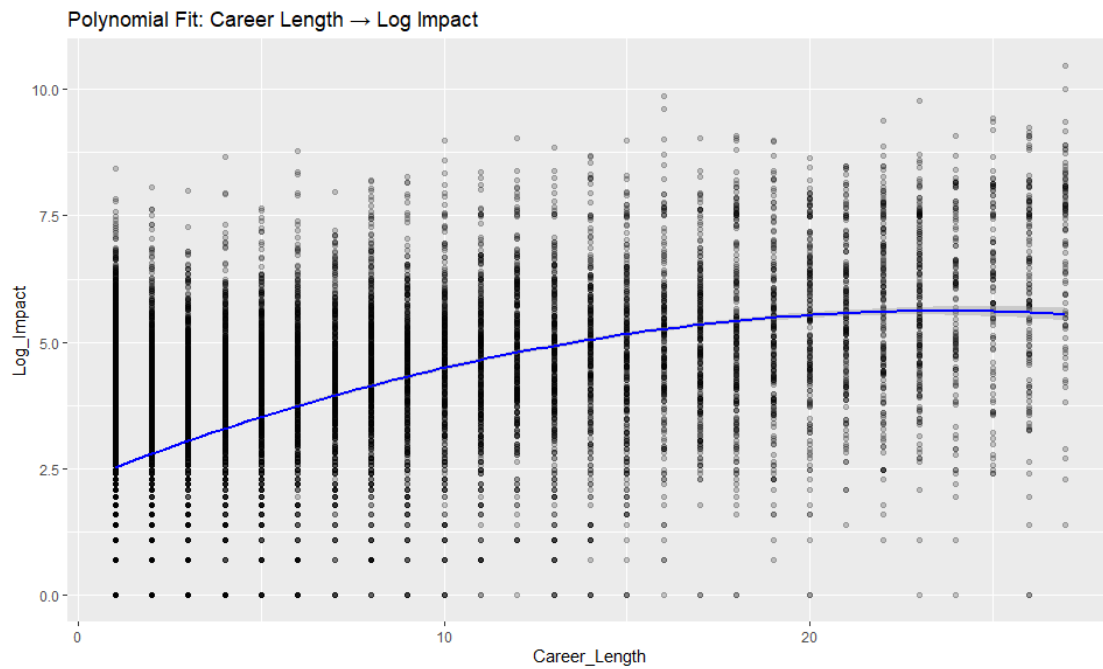


```
# Regular collaborators plot
ggplot(df_small, aes(Num_Regular_Collaborators, Total_Impact)) +
  geom_point(alpha = 0.25, size = 0.7, colour = "tomato") +
  geom_smooth(method = "loess", colour = "red4", linewidth = 1) +
  labs(title = "Regular Collaborators vs Impact (5% Sample)",
       x = "log(Regular Collaborators + 1)",
       y = "Total Impact")
```



A.2: Polynomial Fits

```
# Polynomial Fits
ggplot(df_small, aes(Career_Length, Log_Impact)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", formula = y ~ poly(x, 2), colour = "blue") +
  labs(title = "Polynomial Fit: Career Length → Log Impact")
```



A.3: Misclassification Profile

```
# Misclassification Profile
```

```
# Compute probabilities on df_small
```

```
probs <- predict(log_model, newdata = df_small, type = "response")
```

```
# Assign safely
```

```
df_small$prob <- probs
```

```
# Now classification works
```

```
df_small$pred_class <- ifelse(df_small$prob > 0.5, 1, 0)
```

```
df_small$pred_class <- ifelse(df_small$prob > 0.5, 1, 0)
```

```
df_small$err_type <- case_when(  
  df_small$pred_class == 1 & df_small$Is_HighPerformer == 0 ~ "False  
Positive",  
  df_small$pred_class == 0 & df_small$Is_HighPerformer == 1 ~ "False  
Negative",  
  TRUE ~ "Correct"  
)
```

```
ggplot(df_small, aes(log_Num_Elite, Cohesion_Elite, colour = err_type)) +  
  geom_point(alpha = 0.5) +  
  labs(title = "Misclassification Profile",  
       colour = "Error Type")
```

